UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master's Degree in Computer Science

AMM Scheduler:

Design and Implementation of a Runtime Scheduling System for Asymmetric Heterogeneous Computing

Supervisor:

Prof. Marco Danelutto

Candidate:

Simone Stanganini

ACADEMIC YEAR 2024/2025

Contents

Chapter 1

Introduction

1.1 Contest

Until a few years ago, performance improvements depended mainly on increasing CPU clock speeds. Today the approach has changed completely, shifting the focus towards hardware specialization.

Historically, this specialization was driven by distinct applicative domains. Two of the most common examples are that GPUs were designed for rendering pixels, while DSPs handled only signal processing. However, in recent years, the landscape of high performance computing has witnessed a fundamental change; with the explosive growth of Artificial Intelligence, a large number of modern applications from computer vision to LLMs are impacted by the performance achieved in the execution of a core kernel: **matrix multiplication**.

Consequently, hardware architectures have evolved to optimize this specific operation. Modern computers from servers to embedded SoCs, don't rely on a single processor anymore. Instead, they integrate a heterogeneous suite of execution units, all capable of performing linear algebra, but with different characteristics:

- **CPU**: Optimized for sequential tasks, complex logic and low latency.
- **GPU**: Available as integrated (iGPU) or discrete (dGPU), in a lot of cases both at the same time, designed for massive parallel computing.
- **NPU** (Neural Processing Unit): A recently introduced unit, specialized in speeding up matrix operations for AI workloads, trading flexibility for extreme efficiency.

This variety of hardware has created a gap with the software. While hardware offers different resources for different workloads, traditional software tends to be static. It

usually assigns tasks to a single device, often just the CPU or GPU, and ignores the other available resources.

The result is an inefficient usage of the system. We could get more throughput with the same efficiency, or be more efficient at the same speed.

This is precisely where our project fits in. Since the computational core of modern applications has converged on matrix operations, the current challenge is orchestration: we need a system capable of dynamically mapping these matrix multiplication tasks to the right accelerator, exploiting the entire heterogeneous topology of the system to maximize performance.

1.2 Objectives

The main goal of this thesis is therefore the design and implementation of a scheduler capable of dynamically assigning computational tasks, specifically streams of matrix multiplications, to the different accelerators found in modern architectures.

The work started with an essential preliminary phase, I focused on integrating and studying different compute backends, such as CUDA, SYCL & OpenVINO. This step was useful to create a uniform abstraction layer over the hardware and to verify the technical feasibility of the system.

After this initial phase, the work moved on to the actual development of the scheduler, the core of this project, focused on the asymmetric orchestration logic, designed to decide in real-time which resources will be used for each specific ideal workload such that utilization and efficiency are maximized.

Finally, the system was validated with experimental results on a complex hybrid architecture. This experimental setup included an Intel multicore CPU (with integrated GPU and NPU) and a discrete NVIDIA GPU. The tests used to verify the operation were streams of matrices of the same size, streams of differently sized matrixes, and even splitting larger tasks across multiple accelerators.

1.3 Thesis Structure

The rest of this work is organized as follows:

- **Chapter 2**: Introduces the hardware technologies (CPU, GPU, NPU), the software backends used (CUDA, SYCL, OpenVINO, OpenBLAS) and the software stack used for this project (C, C++, Cmake).
- **Chapter 3**: Describes the logical design, divided into two phases: the creation of the benchmarking environment and the subsequent design of the scheduler.
- **Chapter 4**: Details the code implementation, it covers the wrappers for hardware abstraction, the essential data structures for the scheduler (ringbuffer and performance map), and the techniques used for task splitting, as well as the project compilation procedures and details.
- **Chapter 5**: Analyzes the experimental results, validating the effectiveness of the scheduler on different workloads.
- **Chapter 6**: Draws conclusions on the work done and suggests possible future developments.

Chapter 2

Tools and Technologies

This chapter presents the hardware and software resources used to develop and validate our scheduler. The choice to work on a hybrid and highly heterogeneous architecture was made to test the scheduler in a real-world scenario, and in this scenario, devices with different performance characteristics and programming models have to work together.

2.1 Experimental Platform

All development and benchmarking activities were done on a mobile workstation Lenovo Yoga Pro 9i gen 9. This platform represents a modern high-performance "Edge" computing node well. It integrates all the types of accelerators we considered into a single physical system: multicore CPU, NPU, integrated GPU, and discrete GPU. Having these components available at the same time allowed to emulate, within a single node, the orchestration issues of heterogeneous distributed systems.

2.1.1 The SoC: Intel Core Ultra 9 185H

The core of the system is the Intel Core Ultra 9 185H processor, based on the "Meteor Lake" architecture [2]. This component marks a significant change from traditional monolithic CPUs. It uses a disaggregated design (tiled architecture) that combines different specialized processing units:

Figure 2.1: Intel Ultra Logo

- **Host CPU**: This cpu is made of 16 hybrid cores, 6 Performance-cores, 8 Efficient-cores, and 2 Low-Power-cores for efficiency. In this project this unit acts both as host and accelerator. It runs the scheduler code and does some computation through the OpenBlas library (see Sec. **??**).
- **NPU**: The Ultra 9 includes a native Neural Processing Unit. This is a low-power accelerator designed specifically for neural network inference. Its compute acceleration is facilitated by Neural Compute Engines, which consist of hardware acceleration blocks for AI operations like Matrix Multiplication and Convolution, alongside Streaming Hybrid Architecture Vector Engines for general computing tasks [4]. In this project the NPU is exploited via the OpenVINO (see Sec. 2.4) backend.
- **iGPU**: Intel Arc Graphics, integrated into the SoC. This graphics unit based on Xe-LPG architecture [1], offers parallel computing capabilities with direct access to system memory, in this project this accellerator has been exploited via with the SYCL library (see Sec. 2.3) as the backend to leverage the full performance of this accellerator.

2.1.2 NVIDIA GeForce RTX 4060

Figure 2.2: Nvidia Rtx Logo

To complement the Intel SoC the system is equipped with a discrete NVIDIA GeForce RTX 4060 Laptop GPU. This device is based on the Ada Lovelace architecture [8] and acts as the high-throughput accelerator of the system.

Unlike the integrated GPU this card features 3072 CUDA Cores for general parallel processing and 96 Fourth-Generation Tensor Cores. The Tensor Cores are specifically designed to accelerate dense matrix multiplications with 16 or even 8 bit operations.

The device utilizes 8 GB of GDDR6 VRAM with a 128-bit memory interface, this dedicated memory provides high bandwidth but requires explicit data transfer over the PCIe bus.

Within the scheduler this device is managed via the CUDA backend (see Sec. 2.2).

2.2 CUDA Toolkit

Figure 2.3: Nvidia Cuda Logo

To exploit the computational capabilities of the discrete NVIDIA GPU the system utilizes CUDA (Compute Unified Device Architecture). This parallel computing platform and programming model, developed by NVIDIA, enables dramatic increases in computing performance by harnessing the power of the GPU. It allows developers to accelerate compute- intensive applications using C, C++, and Fortran, and is widely adopted in fields such as deep learning, scientific computing, and more in general high-performance computing (HPC) [10].

In this project CUDA is the tool chosen to extract maximum throughput from the hardware (see Sec. 2.1.2). The implementation relies on specific libraries and mechanisms to optimize the execution of matrix multiplication streams.

- **cuBLAS and Tensor Cores**: to perform linear algebra operations we adopts cuBLAS (CUDA Basic Linear Algebra Subroutines). This library is the GPU-accelerated version of the standard BLAS specification and provides highly optimized implementations of matrix operations, specifically, cuBLAS allows the utilization of Tensor Cores, specialized hardware units designed to accelerate mixed-precision matrix multiplication (Float16 and Int8). This significantly increases the theoretical throughput compared to standard floating-point units [9].

- **CUDA Streams**: To handle the latency introduced by data transfers between the CPU and the GPU the system utilizes CUDA Streams. A stream is a sequence of operations that execute in issue-order on the GPU, by employing multiple streams it is possible to achieve computation/communication overlapping: while one stream is transferring data over the PCIe bus another stream can execute a computational kernel [13]. This mechanism, known as "latency hiding", is fundamental for real-time processing and ensures that the GPU compute units remain active as much as possible.

- **NVIDIA Management Library (NVML)**: For the analysis of energy efficiency the project integrates the NVIDIA Management Library (NVML). This serves as a monitoring tool that provides direct access to the GPU hardware sensors, it allows the retrieval of real-time metrics such as power usage (in milliJoule) and temperature without requiring external physical probes [11].

2.3 SYCL

While CUDA provides maximum performance for NVIDIA hardware, it requires a proprietary programming model, and to address the need for portability and to fully exploit the Intel hardware present in the system, this project adopts the SYCL standard through the Intel oneAPI toolkit.

- **The SYCL Standard**:

Figure 2.4: SYCL Logo

SYCL is an open, royalty-free standard managed by the Khronos Group. It enables high-performance computing on heterogeneous accelerators (such as CPUs, GPUs, and FPGAs) using standard ISO C++ [15]. As defined in the official specification, SYCL provides an abstraction layer that allows developers to write code for heterogeneous processors using a "single-source" style. This means that host code on the CPU and the device code on the accelerator are written in the same file, utilizing standard C++ templates and lambda functions. This approach eliminates the complexity of managing separate source files for different architectures.

- **Intel oneAPI**:

Figure 2.5: OneApi Logo

SYCL requires a specific software development kit (SDK) to be compiled and executed. For this thesis, the Intel oneAPI Base Toolkit was selected [3]. oneAPI is Intel's unified programming model designed to simplify development across diverse architectures. It implements the SYCL standard via the `icpx` compiler, and provides a comprehensive set of libraries optimized for a large set of devices. The choice of this toolkit allows the project to target the available Intel hardware efficiently. Moreover this choice guarantees that the codebase could seamlessly target accellerator from othe vendor like AMD without having to rewrite the code.

2.4 Openvino

Figure 2.6: OpenVino Logo

Developed by Intel, this open-source toolkit is designed to optimize and deploy deep learning models across various types of hardware, from edge devices to cloud servers [5].

The main function provided by the library is that it acts as a translator, it takes an AI model and converts it into a specific format that the target hardware understands. This is very useful because it allows the program to run on different components like the CPU, NPU or the integrated graphics without needing to rewrite the code for each specific accellerator.

The toolkit automatically handles the low-level details to ensure the task runs as fast as possible in the chosen device. This efficiency is achieved through the model optimizer, as described in the documentation, this tool analyzes the computational operations and simplifies them (for example, by merging multiple mathematical steps into one) before execution.

The result is a hardware-agnostic format called intermediate representation, IR, which allows the runtme to map the operations efficiently to the available resources [5].

In this project OpenVINO was essential for targeting the specific NPU found in the system (see Sec.2.1.1). Since the NPU cannot run standard C++ code directly, OpenVINO acts as the bridge between the scheduler and the hardware, specifically, it was implemented to execute Matrix Multiplication tasks on the NPU. This allowed the system to offload these calculations to the neural accelerator, taking advantage of its high energy efficiency.

2.5 OpenBlas

Figure 2.7: OpenBlas Logo

To enable efficient processing on the Host CPU, the project integrates OpenBLAS. OpenBLAS is an optimized, open-source implementation of the BLAS (Basic Linear Algebra Subprograms) API.

Historically, the project emerged as a successor to GotoBLAS2, a highly regarded library developed by Kazushige Goto. After the original development ceased, the OpenBLAS community took over to maintain the code and expand support for modern architectures [16].

The library achieves high performance through a mechanism called runtime processor detection. OpenBLAS first identifies the CPU architecture and automatically selects the most optimized functions for that specific hardware. These functions are often implemented in assembly language to leverage SIMD instructions, such as AVX2 or AVX-512, which allow the processor to handle multiple data points simultaneously (vector computing). Additionally, the library automatically handles multi-threading to distribute the computational load across all available processor cores[12].

In this project this library allows the Host CPU to be treated as an additional accelerator. Rather than limiting the CPU solely to orchestration tasks, OpenBLAS enables the scheduler to use some of the processor cores as a worker node. More details on this will be provided in the Implementation Chapter 4.

2.6 C/C++

Figure 2.8: C and C++ Logo

The C and C++ programming languages represent two of the most influential and widely used tools in the history of computer science. Known for their performance, versatility, and low-level control, they are employed in a vast range of sectors, from operating systems to high-performance applications.

Below is a brief overview of both languages:

The C Language: Developed in the 1970s by Dennis Ritchie, it was one of the first high-level languages in history. It is renowned for its simplicity, efficiency, and direct access to memory, making it ideal for writing highly optimized code [6].

The C++ Language: C++ is an extension of C developed in the 1980s by Bjarne Stroustrup. Among its key features is the support for Object-Oriented Programming (OOP), which allows for the creation of more structured and modular software compared to procedural C [14]. It provides a rich Standard Library (STL) and is the industry standard for performance-critical applications.

In the context of this project, C plays a critical architectural role. It was utilized to define the Application Binary Interface (ABI) between the scheduler and the various hardware wrappers. This choice was driven by the need for a stable and simplified interface. Since the project components are built with different compilers, C++ was unsuitable due to its lack of a standardized ABI, which causes binary incompatibility. C, instead, ensures consistent linkage across these environments. Instead C++ serves as the primary development language. Its adoption was mandatory as it is the native language required by the SDKs of all the utilized frameworks. It is used to implement the complex logic of the scheduler and the data structures necessary for task management. Further details on this will be provided in the Implementation Chapter 4.

2.7 Cmake

Figure 2.9: Cmake Logo

To manage the compilation process of such a complex architecture, the project utilizes CMake (Cross-Platform Make). CMake is an open-source family of tools designed to build, test, and package software. It was originally created by Kitware in 2000 to address the need for a powerful build environment capable of supporting cross-platform development [7].

CMake does not build the software directly, instead it acts as a generator: it reads configuration files named CMakeLists.txt, and generates standard build files for the native toolchain of the operating system, such as Makefiles for Linux or Visual Studio projects for Windows. This abstraction allows developers to describe how the project should be built in a way that is independent of the specific compiler being used.

In this thesis its primary function is to manage the complexity of dynamic linking: since the project depends on large external frameworks (CUDA, OpenVINO, OneAPI, OpenBlass), CMake is configured to locate and link these as shared libraries. This ensures that the executable remains lightweight and that dependencies are resolved correctly at runtime.

Additionally, it enables a modular structure where specific hardware backends can be included or excluded via simple configuration flags.

Chapter 3

Logical Design

This chapter describes the logical structure of the project, the design process was divided into two main phases:

- The first part of the chapter (see Sec. 3.1) presents the Independent Benchmarks. Before developing the centralized scheduler, standalone applications were created for each accelerator. This phase was essential to verify the feasibility of the project and to understand how to effectively control the specific hardware using the selected libraries.
- The second part (see Sec. 3.2) provides a comprehensive overview of the AMM Scheduler. This section describes the high-level operation of all the components of the scheduler, it illustrates the architectural design of the main project and explains how the various logical blocks interact to enable the scheduler's functionality, detailing the specific operational logic that governs the system.

3.1 Preliminary Phase

Before designing the centralized scheduler, the project required a preliminary phase focused on the independent analysis of each software stack. Three standalone applications were developed, targeting the NVIDIA GPU, the Intel iGPU, and the Intel NPU respectively. The primary objective of this phase to gain a deep understanding of the specific libraries (CUDA, SYCL, and OpenVINO) and their interaction with the hardware.

It is important to note that the OpenBLAS backend for the CPU was not included in this preliminary phase. This component was introduced directly during the development of the scheduler (described in the next section 3.2) with a specific goal: to maximize the degree of heterogeneity of the system, by treating the CPU as an additional accelerator rather than just a coordinator. Moreover unlike the GPU and NPU, which required a specific study of their asynchronous drivers and memory models, the CPU integration relies on standard synchronous function calls. Consequently, a dedicated standalone analysis deemed unnecessary. The technical details of this integration will be discussed in the implementation Chapter 4.

3.1.1 Studying the Common Patterns

A crucial goal of this stage was to gain a deep, hands-on understanding of the specific libraries. Since technologies like CUDA, SYCL, and OpenVINO manage hardware in fundamentally different ways, so a first approach phase was necessary to see how their specific mechanisms works and the drivers interactions.

This distinction is particularly evident in the case of OpenVINO. While CUDA and SYCL follow a traditional *kernel launch* paradigm, the NPU is designed exclusively for deep learning graphs, not for general-purpose linear algebra. Consequently, to perform a standard matrix multiplication (GEMM) on this accelerator, a workaround was required; instead of invoking a mathematical function, the operation had to be encapsulated within a synthetic neural network layer, effectively "tricking" the driver into treating a raw calculation that we wanted to perform as an inference task (more on this in section 4.2.2).

Despite these architectural differences, this preliminary study revealed a recurring pattern, the execution flow for every accelerator whether running a kernel or a simulated inference relies on four identical logical steps:

- **Context Initialization**: setting up the driver handles like context, queue, and so on.
- **Memory Management**: allocating buffers and managing host-to-device transfers.
- **Kernel Execution**: triggering the asynchronous computation.
- **Synchronization**: waiting for the hardware to complete the task.

Recognizing this common structure for each library was fundamental for the design of the Abstraction Layer used in the scheduler.

3.1.2 Benchmarking

During this phase, benchmarking routines were implemented within each of the standalone applications. However, their purpose was distinct from the final experimental results presented in later chapters. In this context, the measurements of the execution time and power consumption were intended to act as a development tool to:

- Verify the correctness of the implementation (e.g., ensuring the code was actually running on the accelerator and not falling back to the CPU).
- Analyze the runtime behavior of the drivers, and identify architectural bottlenecks (such as PCIe transfer overheads or compilation latencies).

However, we must point out that the performance profile observed in these isolated environments proved to be quite different from the behavior, observered when using the full scheduler. The concurrent execution of different workloads and the increased stress on system resources exposed new bottlenecks that were not visible during the standalone tests. Consequently, the backend implementations significantly evolved during the integration phase, and specific structures were introduced to mitigate these latencies. These optimizations and the specific issues encountered under high-load scenarios will be analyzed in detail in the Implementation Chapter (4).

3.2 AMM Scheduler

The core of the project is the AMM Scheduler, that stands for Asymettric Matrix Multiplication Scheduler.

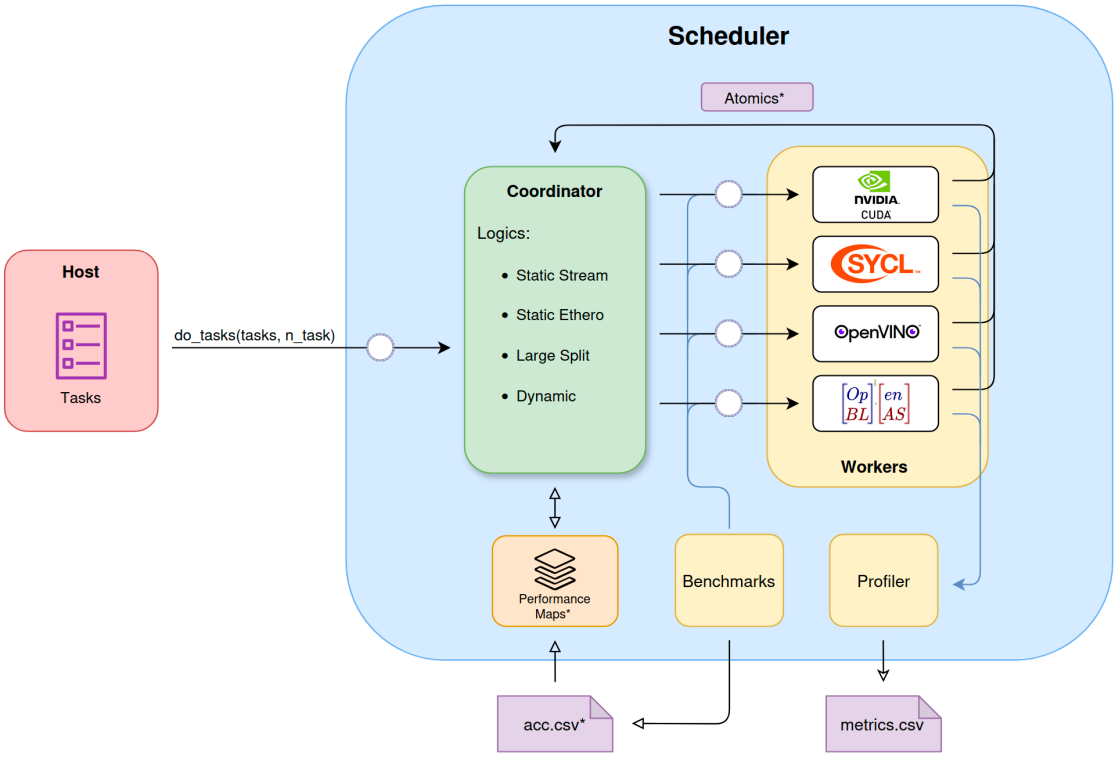

Figure 3.1: AMM Scheduler Components

As illustrated in Fig. 3.1, the architecture implements a producer and consumer pattern.

In this model, the host application acts as the producer, it generates computational tasks for the scheduler, that behave as the consumer, processing these requests asynchronously. The execution logic follows the farm pattern: a central coordinator retrieves the tasks and distributes them to the available worker nodes. To enable parallel execution, the coordinator and each worker operate as independent threads.

As shown in the diagram, Workers communicate with the Coordinator using atomic variables. This mechanism signals when a Worker is idle or updates the count of completed tasks, this ensures that the scheduler's wait function can correctly determine when all tasks are finished.

Now we will examine each component in the diagram and explain a bit deeper its bahvior:

Figure 3.2: Ring Buffer Rappresentation

- **Ring Buffers**: The key to this asynchronous pattern is the ring buffer (rappresented in the Fig. 3.2 as the arrows marked with a circle), which decouples the Host from the Coordinator, designed ad-hoc for the scheduler. This structure enables extremely fast communications with minimal overhead, using atomic variables instead of locks. As the graph in Fig 3.1 illustrates, this mechanism also implements the communication between the Coordinator and the Workers, minimizing the overhead and maximizing speed during task distribution. More on the code implementation in Sec. **??**.

Figure 3.3: Coordinator

- **Coordinator**: The Coordinator (represented in Fig. 3.3 acts as the brain of the scheduler. It decides how to dispatch tasks and implements different strategies to demonstrate various resource management techniques.

  It features distinct scheduling logics, based on the type of workload that the host have to compute. The main logics that shape the design of the scheduler are the static ones.

  The first logic, Static stream, handles static homogeneous workloads (tasks of identical matrices). By using a Performance Map (a custom data structure containing heuristics and execution time estimates) the coordinator predicts how long a task will take on a specific accelerator and based on this, it performs a static dispatch of the tasks in a batch style way (see Sec. 4.3.9, Static Partitioning)

  The second logic, Static Hetero, work in a similar way but for static heterogeneous workloads (matrices of different sizes), the coordinator uses the same performance map to calculate the optimal distribution, ensuring the workload is balanced across all accelerators (see Sec. 4.3.11, Static Hetero Partitioning).

To evaluate these strategies, we also implemented a dynamic tasks dispatch logic. This approach assigns tasks to the first available Worker (i.e. idle) and serves as a baseline to verify the efficiency of our static methods (see Sec. 4.3.12, Dynamic).

Finally, the Coordinator supports the Task Splitting logic. If a matrix exceeds the memory capacity of a single accelerator, the logic breaks it down into smaller sub-tasks such that they may be distributed across multiple accellerators (see Sec. 4.3.10, Large Matrix Split).

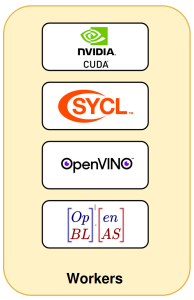

Figure 3.4: Workers

- **Workers**: Workers (represented in Fig. 3.4) are threads that utilize the abstraction layers defined in the preliminary phase (see Sec. 3.1). Their internal logic is straightforward: they retrieve available tasks and execute them on their assigned accelerator. All low-level details regarding the accelerator libraries are completely hidden from the Worker prospective (see Sec. 4.3.2, Workers).

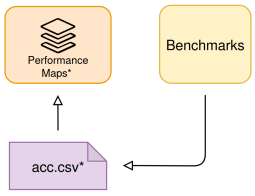

Figure 3.5: Benchmark and Performancemaps Rapresentation

- **Benchmarks and Performance Maps**: Internally, the scheduler utilizes a data structure called the Performance Map (represented in Fig. 3.5) to determine the execution time of every single task for each individual accelerator. The structure contains heuristics and data loaded from CSV files (specifically one file for each accelerator). These files are created by a benchmark routine developed directly within the scheduler. If the csv for one or more accelerators is missing so that there are no data for the performance maps, the routine activates automatically to generate the necessary information stored in the csv files, it utilizes the accelerator libraries, testing them by executing precise tasks in a specific way to gather this data as accurately as possible. Furthermore, the

Performance Map is an extremely fast structure: it is equipped with caching and highly optimized internal data structures to minimize overhead on the Coordinator's logic, which is the only component that accesses this structure (see Sec. 4.3.6, Performance Map).

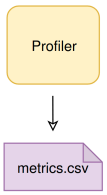

Figure 3.6: Profiler

- **Profiler**: To conduct an extensive and precise study of how the scheduler behaves across all workloads, a component called the Profiler (represented in Fig. 3.6) was developed directly within the scheduler. It is responsible for profiling all operational strategies and recording all execution times for both the scheduler and the workers. It returns all these metrics in an organized and readable manner in the form of CSV file. These metrics include the power consumption of the entire system and of each individual accelerator, as well as the specific overhead for every single phase of the scheduler's logic and a lot of more information that will be described later (see Sec. 4.3.7, Profiler).

Chapter 4

Implementation

In this chapter, after having seen a general explanation of how the scheduler works in the previous sections, we are going to look at and explain all its components in detail.

In the first section, we will explain the structure of the project files and the reason why they are organized this way, and we will also give a view of the entire compilation process of the scheduler.

Subsequently, we will explain each of the abstraction layers used to target the accelerators, and then to conclude, we will see in detail all the components that make the scheduler logic work.

As mentioned in Chap. 3, logical design, the code developed during the preliminary phase is not analyzed in this section. This avoids repetition, as the evolved version of this code, which is integrated into the scheduler, will be explained in detail later. However, all the original code is available in Appendix **??**.

4.1 Project Structure and Compilation

The figure below provides a view of the project's file and folder structure:

Project Tree

```
|-- bin/
|-- build/
|-- cuda/
|   |-- runner.cu
|   |-- cuda_wrapper.h
|   |-- CMakeLists.txt
|-- sycl/
|   |-- runner.cpp
|   |-- sycl_wrapper.h
|   |-- CMakeLists.txt
|-- openvino/
|   |-- runner.cpp
|   |-- ov_wrapper.h
|   |-- CMakeLists.txt
|-- scheduler/
|   |-- main.cpp
|   |-- scheduler.hpp
|   |-- performancemap.hpp
|   |-- sharedbuffer.hpp
|   |-- cpu.hpp
|   |-- profiler.hpp
|   |-- tasks.hpp
|   |-- tests.hpp
|   |-- config.hpp
|   |-- CMakeLists.txt
|-- CMakeLists.txt
|-- make_run.sh
|-- merge_clangd.py
|-- setup.txt
```

Figure 4.1: Project Dir Tree

The codebase of this project was designed with a specific goal: modularity. Since the project works with very different hardware architectures, writing all the code in a single environment is not feasible. Each accelerator requires its own specific compiler and libraries: NVIDIA GPUs need the `nvcc` compiler, Intel GPUs need the `icpx` compiler for SYCL, and the NPU relies on OpenVINO libraries.

To solve this compatibility issue, the architecture was decomposed into independent dynamic libraries. Each hardware backend (CUDA, SYCL, OpenVINO) is compiled into a separate shared object file (`.so`), and each of these libraries expose a standard C ABI (Application Binary Interface). This design is mandatory because the C ABI acts as a universal language that allows the main scheduler to communicate with the accelerators without needing to understand their specific C++ implementations or complex driver dependencies, moreover the real problem was with the version of the compilers utilized with each specific library, so the standard approach is to unify all the interfaces with the help of the C language.

As shown in Fig. 4.1, the project includes a dedicated folder for each library (`sycl/`, `openvino/`, and `cuda/`) containing its specific implementations, finally, the `scheduler/` directory containing all the internal dependencies used by the scheduler logic, which will be explained in Sec. 4.3.

4.1.1 Compilation Process

The tool CMake was used in this project to manage this complex build process, the build is not performed all at once.

Below is the entire CMakeLists.txt file in the root directory:

```
cmake_minimum_required(VERSION 3.15)
project(amm_scheduler)

include(ExternalProject)

set(LIB_OUTPUT_DIR ${CMAKE_SOURCE_DIR}/bin)
file(MAKE_DIRECTORY ${LIB_OUTPUT_DIR})

# CUDA SYCL OPENVINO
option(USE_CUDA "CUDA ON" OFF)
option(USE_SYCL "SYCL ON" OFF)
option(USE_OPENVINO "OpenVINO ON" OFF)
option(USE_OPENBLAS "OpenBLAS ON" OFF)
option(USE_OPENCV "OpenCV ON" OFF)

set(SCHEDULER_DEPS "") # depends variable

set(OPT_FLAGS "-O2 -DNDEBUG")

# CUDA
if(USE_CUDA)
    message(STATUS "CUDA Backend ON")
    ExternalProject_Add(build_cuda
```

```
        SOURCE_DIR ${CMAKE_SOURCE_DIR}/cuda
        CMAKE_ARGS
            -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
            -DCMAKE_BUILD_TYPE=Release
            -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
            -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}
            -DCMAKE_CUDA_FLAGS_RELEASE=${OPT_FLAGS}
            #BUILD_ALWAYS 1
    )

    list(APPEND SCHEDULER_DEPS build_cuda)
else()
    message(STATUS "CUDA Backend OFF")
endif()

# SYCL and OPENVINO are the same as cuda

# scheduler
ExternalProject_Add(build_scheduler
    SOURCE_DIR ${CMAKE_SOURCE_DIR}/scheduler
    CMAKE_ARGS
        -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
        -DCMAKE_BUILD_TYPE=Release
        -DLIBS_DIR=${LIB_OUTPUT_DIR}
        -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}

        -DCUDA_SRC_DIR=${CMAKE_SOURCE_DIR}/cuda
        -DSYCL_SRC_DIR=${CMAKE_SOURCE_DIR}/sycl
        -DOPEN_SRC_DIR=${CMAKE_SOURCE_DIR}/openvino

        -DENABLE_CUDA=${USE_CUDA} # USE_CUDA
        -DENABLE_SYCL=${USE_SYCL} # USE_SYCL
        -DENABLE_OPENVINO=${USE_OPENVINO} # USE_OPENVINO
        -DENABLE_OPENBLAS=${USE_OPENBLAS} # USE_OPENBLAS
        -DENABLE_OPENCV=${USE_OPENCV} # USE_OPENCV

        -DCMAKE_EXPORT_COMPILE_COMMANDS=ON # for clangd
    DEPENDS ${SCHEDULER_DEPS}
    #BUILD_ALWAYS 1
)

add_custom_target(UpdateLSP
    ALL
    COMMAND python3 ${CMAKE_SOURCE_DIR}/merge_clangd.py
    DEPENDS build_scheduler
)
```

Listing 4.1: CMakeLists.txt root directory

As the code in the Fig. 4.1 show, the `ExternalProject_Add` feature of cmake is utilized to keep the compilation steps isolated. The main `CMakeLists.txt` file in the root directory acts as a manager. It does not compile the code directly, instead, it checks the configuration flags (such as `USE_CUDA`, `USE_SYCL` and `USE_OPENVINO`) and triggers a separate build process for each active module. The process is the following:

1. First, CMake enters the accelerators sub-directories, `cuda/`, `sycl/`, `openvino/`,

and builds them as standalone projects.

2. Each sub-project uses its specific compiler to generate a dynamic library, e.g., `libcuda_lib.so`, `libsycl_lib.so`).

3. Finally, the `scheduler/` directory is compiled. Because of the standard C ABI, the scheduler does not need to know how the hardware works or which compiler was used for the accelerators, it simply links against these shared libraries at runtime.

This separation ensures that the specific code for one hardware component does not interfere with the others. For example, the scheduler compiler does not need to handle CUDA Kernels, but rather it only sees standard C functions.

Based on the flags passed to CMake during compilation, specific preprocessor definitions are passed to the C++ code. This allows the program to enable or disable dependencies on specific libraries directly inside the source code.

Consequently, the scheduler logic knows exactly which accelerators are available. This approach makes the system flexible, if a specific accelerator or library is missing on the machine where the project has to be compiled, it can be disabled simply by using a flag.

Below the following snippet from the scheduler's build configuration shows how the flags trigger the injection of preprocessor definitions and the linking of specific libraries:

```
# CUDA
if(NOT DEFINED ENABLE_CUDA)
    message("[SCHEDULER] CUDA off on scheduler")
    set(ENABLE_CUDA OFF)
endif()

# SYCL
if(NOT DEFINED ENABLE_SYCL)
    message("[SCHEDULER] SYCL off on scheduler")
    set(ENABLE_SYCL OFF)
endif()

# OPENVINO
if(NOT DEFINED ENABLE_OPENVINO)
    set(ENABLE_OPENVINO OFF)
endif()

#OPENBLAS
if(NOT DEFINED ENABLE_OPENBLAS)
    message("[SCHEDULER] OpenBLAS off on scheduler")
    set(ENABLE_OPENBLAS OFF)
endif()
```

Listing 4.2: Part of CMakeLists.txt inside the scheduler directory

The command `target_compile_definitions` passes a macro (e.g., `ENABLE_CUDA`) to the compiler, which the C++ code effectively uses to enable or disable code blocks via `#ifdef` directives. Simultaneously, `target_link_libraries` ensures that the scheduler only links against the dynamic libraries that are actually present.

The compilation process for the accelerator backends follows a standard pattern. As an example, the configuration file for the OpenVINO library is presented in Listing 4.3:

```
cmake_minimum_required(VERSION 3.20)
project(OpenVinoPart LANGUAGES CXX)

message("\n--------------- OPENVINO COMPILING \n")

find_package(OpenVINO REQUIRED)

add_library(ov_lib SHARED runner.cpp)

set_target_properties(ov_lib PROPERTIES PUBLIC_HEADER ov_wrapper.h)

# C++17 needed OpenVINO
target_compile_features(ov_lib PRIVATE cxx_std_17)

target_link_libraries(ov_lib PRIVATE openvino::runtime)

install(TARGETS ov_lib
    LIBRARY DESTINATION lib
    PUBLIC_HEADER DESTINATION include
)
```

Listing 4.3: CMakeLists.txt inside OpenVINO directory

First this script locates the necessary dependencies on the host system using `find_package`, then, it defines a shared library named `ov_lib` using the source file `runner.cpp`. The `SHARED` keyword is fundamental because it instructs the compiler to generate a dynamic library instead of a standard executable.

Finally, the script links the specific runtime (in this case, `openvino::runtime`) and specifies the installation rules. The `install` command ensures that the resulting binary and the public header (`ov_wrapper.h`) are placed in the correct output directories, making them accessible to the main scheduler.

The CUDA and SYCL modules use an identical structure, differing only in the specific compilers and libraries linked.

Finally, when the compilation finishes, all the resulting files are placed in a `bin/` directory. This folder contains the final executable named scheduler, the csv that the scheduler will create at runtime, and all the dynamic libraries it needs.

To speed up the development process, a script named `make_run.sh` was created. This script automates the entire workflow that consist of cleaning the bin and build dir, runs CMake with the correct options to enable all accelerators, compiles the

project using all available CPU cores, and finally executes the program.

Additionally, the project includes a Python script named `merge_clangd.py`. Since multiple independent builds are used, the LSP (Language Server Protocol) of the code editing tools often don't work with these many libraries. This script collects the compilation commands from all the different modules and merges them into a single file, ensuring that code completion and error highlighting work correctly across the entire project. This file will be executed after all the compilation steps are done.

4.2 Accelerators Abstraction Layers

Before diving into the architectural details in the following sections, it is important to clarify the supported data types. The scheduler is fully capable of managing both 32-bit and 16-bit matrix operations. Furthermore, thanks to the modular and flexible design of the codebase, the architecture is already capable to support 8-bit matrices as well. However, as 8-bit operations are not supported by all target accelerators and were not required to achieve the objectives of this thesis, their implementation has not been finalized across all the different backends.

4.2.1 Cuda

The CUDA abstraction layer is designed to manage the NVIDIA discrete GPU available in the system. As previously discussed, the interaction between the C++ scheduler and the NVIDIA compiler (`nvcc`) is mediated by a C interface.

The header file `cuda_wrapper.h` defines the Application Binary Interface exposed to the scheduler (see Listing 4.4). It declares the necessary functions for initialization, memory cleanup, and the execution of matrix multiplication across different data types, 32-bit floating point, 16-bit floating point, and 8-bit integer.

```
#ifdef __cplusplus
extern "C" {
#endif

    void cuda_init();
    void cuda_free();
    void cuda_gemm_32bit(void* A, void* B, void* C, int M, int N, int K);
    void cuda_gemm_16bit(void* A, void* B, void* C, int M, int N, int K);
    void cuda_gemm_8bit(void* A, void* B, void* C, int M, int N, int K);

#ifdef __cplusplus
}
#endif
```

Listing 4.4: cuda_wrapper.h interface

The implementation of these functions is in the `runner.cu` file, but before analyzing the initialization and execution logic, we will examine the global data structures and helper macros defined in the file, these elements manage the persistent state of the device and ensure error reporting.

```
#define CHECK_CUDA(func) {                \
    cudaError_t e = (func);               \
    if(e != cudaSuccess)                  \
        printf ("%s %d CUDA ERROR: %s\n", __FILE__,  __LINE__,
                cudaGetErrorString(e)); \
}
```

```
7
8  #define CHECK_CUBLAS(func) {                          \
9      cublasStatus_t e = (func);                        \
10     if(e != CUBLAS_STATUS_SUCCESS)                    \
11         printf ("%s %d CUBLAS ERROR: ", __FILE__, __LINE__); \
12 }
13 #define N_STREAM 2
14
15 cublasHandle_t handle;
16 cudaStream_t streams[N_STREAM];
17 void *d_A[N_STREAM];
18 void *d_B[N_STREAM];
19 void *d_C[N_STREAM];
20 int i = 0; /* stream id*/
```

Listing 4.5: cuda_runner.cu data and helper

To ensure that the execution progresses without errors, the code utilizes two macros: `CHECK_CUDA` and `CHECK_CUBLAS` (see Listing 4.5). These are used to wrap every API call to the CUDA runtime and the cuBLAS library, respectively. If an operation fails the macro immediately prints the specific error code along with the file name and line number where the failure occurred.

As for as state management is concerned, the system relies on CUDA Streams, a macro `N_STREAM` defines the number of parallel execution queues. Streams, enable the GPU to allows the overlap of memory transfers with computation, or the concurrent execution of multiple kernels. To support this concurrency, the device resources are replicated for each stream. The memory pointers (`d_A`, `d_B`, `d_C`) are defined as arrays, meaning that each stream possesses its own independent pre-allocated memory buffers. Finally, a global integer `i` acts as counter, used to cycle through the available streams when dispatching new tasks.

The design focuses on minimizing the runtime overhead by performing expensive operations only once during the startup. The `cuda_init` function is responsible for preparing the GPU environment before any task is processed. Its primary goal is to eliminate the latency associated with running the dynamic memory allocation istrunctions. Allocating memory on the GPU using `cudaMalloc` is a slow operation compare to the others, infact if the system were to allocate and free memory for every single matrix multiplication task, the overhead would significantly impact the performance. To solve this, the initialization function adopts a simple pre-allocation strategy.

In Listing 4.6 the implementation of the `cuda_init` function:

```
void cuda_init() {
    CHECK_CUBLAS(cublasCreate(&handle));

    /* reset stream counter */
    i = 0;

    size_t MAX = (size_t)4096 * 4096 * sizeof(float);

    for (int j = 0; j < N_STREAM; ++j) {
        CHECK_CUDA(cudaStreamCreate(&streams[j]));
        CHECK_CUDA(cudaMalloc(&d_A[j], MAX));
        CHECK_CUDA(cudaMalloc(&d_B[j], MAX));
        CHECK_CUDA(cudaMalloc(&d_C[j], MAX));
    }
}
```

Listing 4.6: cuda_runner.cu init

At startup, the function first initializes the cuBLAS library handle, which is required to invoke the linear algebra routines. Then, it iterates through `N_STREAM` number of streams (simply 2 stream is enough). For each stream, it creates a CUDA stream object to manage execution queues and allocates three distinct memory buffers on the device (`d_A`, `d_B`, `d_C`). The size of these buffers is calculated to hold the largest matrix dimension supported by the system (MAX size), ensuring that any incoming task fits within the pre-allocated space without requiring reallocation.

It is important to observe that, although the hardware accelerator supports dimensions exceeding this limit, a maximum square matrix size of 4096×4096 was established (for each accelerator). This design choice was made to avoid introducing unnecessary architectural complexity, since it wasn't in this thesis objectives to maximize the memory saturation, and this limit provides a sufficient workload to stress the scheduler without incurring into the typical limitations due to the memory management layer.

Complementary to the initialization, the `cuda_free` (see Listing 4.7) function ensures the proper release of all GPU resources when the scheduler shuts down.

```
void cuda_free() {
    for (int j = 0; j < N_STREAM; ++j) {
        CHECK_CUDA(cudaFree(d_A[j]));
        CHECK_CUDA(cudaFree(d_B[j]));
        CHECK_CUDA(cudaFree(d_C[j]));
        CHECK_CUDA(cudaStreamDestroy(streams[j]));
    }
    CHECK_CUBLAS(cublasDestroy(handle));
}
```

Listing 4.7: cuda_runner.cu free

The function iterates through the created streams, freeing the device memory buffers using `cudaFree` and destroying the stream objects with `cudaStreamDestroy`. Finally, it destroys the global cuBLAS handle to release the library resources.

To maintain compatibility between the C ABI and CUDA C++ features, the execution logic is split into two levels, since the public C interface cannot recognize CUDA specific types it relies on generic `void*` pointers.

To handle this the system implements a layer of simple bridge functions, these functions accept the generic pointers and cast them to the correct concrete type (e.g., `float*` or `__half*`), and immediately forward the call to the internal function (see Listing 4.8).

```
void cuda_gemm_32bit(void* A, void* B, void* C, int M, int N, int K) {
    cuda_gemm_32bit_p((float*)A, (float*)B, (float*)C, M, N, K);
}

void cuda_gemm_16bit(void* A, void* B, void* C, int M, int N, int K) {
    cuda_gemm_16bit_p((__half*)A, (__half*)B, (__half*)C, M, N, K);
}
```

Listing 4.8: cuda_runner.cu type bridging functions

The actual computational logic is implemented in the private functions, such as `cuda_gemm_32bit_p`. This functions orchestrates the entire lifecycle of a single GPU task. (see Listing 4.9)

```
void cuda_gemm_32bit_p(float* A, float* B, float* C, int M, int N, int K) {
        cublasSetStream(handle, streams[i]);

        CHECK_CUDA(cudaMemcpyAsync(d_A[i],
            A, M * K * sizeof(float), cudaMemcpyHostToDevice, streams[i]));
        CHECK_CUDA(cudaMemcpyAsync(d_B[i],
            B, K * N * sizeof(float), cudaMemcpyHostToDevice, streams[i]));

        float a = 1.0f, b = 0.0f;

        cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
                 N, M, K,
                 &a,
                 d_B[i], CUDA_R_32F, N,
                 d_A[i], CUDA_R_32F, K,
                 &b,
                 d_C[i], CUDA_R_32F, N,
                 CUBLAS_COMPUTE_32F,
                 CUBLAS_GEMM_DEFAULT_TENSOR_OP);

        CHECK_CUDA(cudaMemcpyAsync(C,
            d_C[i], M * N * sizeof(float),
            cudaMemcpyDeviceToHost, streams[i]));
        /* blocking for latency metrics */
        CHECK_CUDA(cudaStreamSynchronize(streams[i]));
        /* change stream for the next call */
        i = (i + 1) % N_STREAM;
```

```
}
```

Listing 4.9: cuda_runner.cu 32-bit logic

The function begins by associating the global cuBLAS handle with the current stream (`streams[i]`), this ensures that all subsequent library calls are queued into this specific stream.

The execution begins by asynchronously copying the inputs A and B to the GPU buffers using the `cudaMemcpyAsync`, next, the matrix multiplication is triggered via `cublasGemmEx`, this function is chosen over standard BLAS calls because it offers more flexibility for high-performance computing.

It enables the `CUBLAS_GEMM_DEFAULT_TENSOR_OP` flag, explicitly instructing the GPU to utilize tensor cores whenever is possible (with 16 bit and 8 bit, in more modern GPUs from the Ampere architecture, even 32 bit).

Notably, the input operands are swapped in the function call. This operation ensure the matematical correctness compensating the layout difference between C++ and cuBLAS that are using two different storing method of the matrix, respectively row-major and column-major. With this simple method the function multiply two matrices without requiring an expensive physical transpose

Once computed, the result is copied back to the host. The process concludes with `cudaStreamSynchronize`, which blocks the thread to ensure data validity and accurate timing, followed by an update of the stream index for the next stream.

In listing 4.10 we outline the implementation for the 16-bit cuda gemm:

```
void cuda_gemm_16bit_p(__half* A, __half* B, __half* C,
                       int M, int N, int K) {
    cublasSetStream(handle, streams[i]);

    CHECK_CUDA(cudaMemcpyAsync(d_A[i], A,
        M * K * sizeof(__half), cudaMemcpyHostToDevice, streams[i]));
    CHECK_CUDA(cudaMemcpyAsync(d_B[i], B,
        K * N * sizeof(__half), cudaMemcpyHostToDevice, streams[i]));

    float a = 1.0f, b = 0.0f;

    cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
                 N, M, K,
                 &a,
                 d_B[i], CUDA_R_16F, N,
                 d_A[i], CUDA_R_16F, K,
                 &b,
                 d_C[i], CUDA_R_16F, N,
                 CUBLAS_COMPUTE_32F,
                 CUBLAS_GEMM_DEFAULT_TENSOR_OP);

    CHECK_CUDA(cudaMemcpyAsync(C, d_C[i],
        M * N * sizeof(__half), cudaMemcpyDeviceToHost, streams[i]));
    CHECK_CUDA(cudaStreamSynchronize(streams[i]));
```

```
25
26     i = (i + 1) % N_STREAM;
27 }
```

Listing 4.10: cuda_runner.cu 16-bit logic

The logic stay the same of the 32 bit version, but it introduces changes to handle half-precision data, first the pointer types are replaced with the specific `__half` type, consequently, the `cublasGemmEx` function is configured with the `CUDA_R_16F` flag for the input and output data. However, it is worth pointing out that the computation type is kept as `CUBLAS_COMPUTE_32F`. This is a standard practice to ensures that while the data storage is compressed to 16 bit. The internal mathematical computation happens in 32 bit to prevent overflows.

The implementation for 8 bit integer matrices follows the exact same architectural pattern, differing only in data types, `int8_t` and flags `CUDA_R_8I`. The code can be found in full in Appendix **??**.

4.2.2 OpenVINO

The OpenVINO abstraction layer is designed to target the Intel NPU, just like the CUDA backend, the interaction between the C++ scheduler and the OpenVINO runtime is mediated by a standard C interface to ensure the ABI compatibility.

```
1  #ifdef __cplusplus
2  extern "C" {
3  #endif
4
5      void ov_init();
6      void ov_gemm_32bit(void* A, void* B, void* C, int M, int N, int K);
7      void ov_gemm_16bit(void* A, void* B, void* C, int M, int N, int K);
8      void ov_gemm_8bit(void* A, void* B, void* C, int M, int N, int K);
9      void ov_free();
10
11 #ifdef __cplusplus
12 }
13 #endif
```

Listing 4.11: ov_wrapper.h interface

The header file `ov_wrapper.h` (see Listing 4.11) defines the interface exposed to the scheduler. It's the identical structure used for the other accelerators, declaring functions for initialization, cleanup, and matrix multiplication for 32-bit, 16-bit, and 8-bit data types.

The implementation resides in the `runner.cpp` file. While the function signatures are similar to the CUDA implementation, the internal logic is fundamentally different due to how OpenVINO function, unlike CUDA, which have all pre-compiled

kernels, OpenVINO works by building a so call computational graph and compiling it for the specific target device at runtime.

Compiling a model is an expensive and time consuming operation, it cannot be performed for every single task. To solve this, the system implements a simple caching mechanism.

```
ov::Core core;

using namespace std;

/* cache for compiled models */
static std::map<tuple<int, int, int>, ov::InferRequest> cache_32bit;
static std::map<tuple<int, int, int>, ov::InferRequest> cache_16bit;
static std::map<tuple<int, int, int>, ov::InferRequest> cache_8bit;

const size_t MAX_MAP_SIZE = 20;
size_t map_size_32bit=0;
size_t map_size_16bit=0;
size_t map_size_8bit=0;
```

Listing 4.12: ov runner.cpp chaching

The global variable `core`(see Listing 4.12), represents OpenVINO runtime, it will be utilized for discovering available devices and compiling the models.

To implement the caching mechanism, the system utilizes standard C++ maps for associating a specific set of matrix dimensions (represented by a tuple (M,N,K)) with a already compiled ov::InferRequest. This means that if the scheduler requests a matrix multiplication with dimensions that have been seen before, the system can retrieve the pre-compiled request immediately, skipping the expensive compilation step for already seen matrices.

Finally, to manage memory usage and prevent the cache from growing indefinitely in case there are many different matrices, a simple eviction policy is enforced using the `MAX_MAP_SIZE` constant and associated counters and if the number of cached models exceeds this limit, the cache is cleared to make room for new entries.

```
void ov_init_p() {
    bool available = false;

    for (auto &d : core.get_available_devices())
        if (d.find("NPU") != string::npos)
            available = true;

    if (!available) {
        cout << "[OPENVINO] NPU not present, aborting \n";
        exit(EXIT_FAILURE); /* shut down something is wrong */
    }
}
```

Listing 4.13: ov runner.cpp init

The init function simply queries the OpenVINO Core to verify the presence of the NPU device. If the hardware is not found, the system aborts to prevent runtime failures during execution.

The core logic of the OpenVINO backend is found in the caching functions, such as `cache_32bit`. This function is responsible for constructing the neural network graph that represents a matrix multiplication.

```
void cache_32bit(tuple<int, int, int> key) {
    if (cache_32bit.find(key) == cache_32bit.end()) {
        map_size_32bit++;
        if (map_size_32bit >= MAX_MAP_SIZE) {
            cache_32bit.clear();
            map_size_32bit = 1;
        }
        auto A_ov = std::make_shared<ov::op::v0::Parameter>
            (ov::element::f32, ov::Shape{(size_t)get<0>(key),
            (size_t)get<2>(key)});
        auto B_ov = std::make_shared<ov::op::v0::Parameter>
            (ov::element::f32, ov::Shape{(size_t)get<2>(key),
            (size_t)get<1>(key)});

        auto matmul = std::make_shared<ov::op::v0::MatMul>(A_ov, B_ov);
        auto result = std::make_shared<ov::op::v0::Result>(matmul);

        auto model = std::make_shared<ov::Model>(result,
                                    ov::ParameterVector{A_ov, B_ov});

        ov::CompiledModel compiled_model = core.
                                        compile_model(model, "NPU");
        ov::InferRequest request = compiled_model.create_infer_request();

        cache_32bit[key] = request;
    }
}
```

Listing 4.14: ov runner.cpp caching function

The graph construction consist in three steps, first two input nodes are created with the specific shape and precision in this case 32-bit float. Second, a `MatMul` node is instantiated taking these parameters as inputs, defining the GEMM Operation. Finally, the graph is wrapped into an `ov::Model` object and compiled for the NPU device. This step optimizes the graph for the specific hardware architecture, and the resulting request object is stored in the cache map.

Once the model is cached, the execution of the task is handled by the `ov_gemm_32bit_p` function.

```
void ov_gemm_32bit_p(void *A, void *B, void *C, int M, int N, int K){
    auto key = std::make_tuple(M, N, K);
    cache_32bit(key);

    ov::InferRequest& infer_request = cache_32bit[key];

```

```
7       ov::Tensor tensor_A(ov::element::f32, ov::Shape{(size_t)M,
8                                                       (size_t)K}, A);
9       ov::Tensor tensor_B(ov::element::f32, ov::Shape{(size_t)K,
10                                                      (size_t)N}, B);
11      ov::Tensor tensor_C(ov::element::f32, ov::Shape{(size_t)M,
12                                                      (size_t)N}, C);
13
14      infer_request.set_input_tensor(0, tensor_A);
15      infer_request.set_input_tensor(1, tensor_B);
16      infer_request.set_output_tensor(0, tensor_C);
17
18      infer_request.infer();
19  }
```

Listing 4.15: ov runner.cpp gemm 32 bit

This function first calls the cache function to ensure the request object is available. Then, it wraps the raw pointers provided by the scheduler (A, B, C) into `ov::Tensor` objects. Notably this constructor uses the existing memory pointers without allocating new buffers or copying data (zero-copy). Finally, the tensors are bound to the input/output ports of the model, and the `infer()` method is called to trigger the execution on the NPU.

Finally, the code exposes the public C ABI by wrapping the internal C++ functions. This structure mirrors the approach used in the CUDA backend, ensuring that the scheduler sees a uniform interface across all accelerators.

```
1  extern "C" {
2      void ov_init() {
3          ov_init_p();
4      }
5
6      void ov_gemm_32bit(void *A, void *B, void *C, int M, int N, int K){
7          ov_gemm_32bit_p(A, B, C, M, N, K);
8      }
9
10     void ov_gemm_16bit(void *A, void *B, void *C, int M, int N, int K){
11         ov_gemm_16bit_p(A, B, C, M, N, K);
12     }
13
14     void ov_gemm_8bit(void *A, void *B, void *C, int M, int N, int K){
15         ov_gemm_8bit_p(A, B, C, M, N, K);
16     }
17     void ov_free() {
18     }
19 }
```

Listing 4.16: ov runner.cpp wrappers

A notable difference compared to the CUDA implementation is the `ov_free` function, which is empty. This is because the OpenVINO runtime relies on modern C++ features, specifically RAII and smart pointers, so when the program terminates or the objects go out of scope, the destructors for the `ov::Core` and the cached requests

are invoked automatically, releasing the NPU resources explicit cleanup.

4.2.3 SYCL

The SYCL abstraction layer is designed to manage Intel GPUs. In this project, SYCL is utilized in conjunction with the Intel oneAPI Math Kernel Library (oneMKL) to perform high-performance matrix operations.

Just like the previous modules, the interaction with the scheduler is mediated by a pure C interface to ensure ABI compatibility. The header file `sycl_wrapper.h` defines the standard functions for initialization, execution (32-bit and 16-bit), and cleanup.

The implementation logic begins with the initialization phase (see Listing 4.17):

```
void init() {
    try {

        /* persisten streams */
        for(int i=0; i<N_STREAMS; i++) {
            streams.push_back(std::make_unique<SYCLStream>(MAX));
        }

    } catch (sycl::exception const& e) {
        std::cerr << "[SYCL ERROR] Init: " << e.what() << "\n";
        exit(1);
    }
}
```

Listing 4.17: sycl runner.cpp init

The primary task of the `init` function is to populate the global vector named `streams` with instances of `SYCLStream`. By iterating `N_STREAMS` times, it creates multiple independent execution contexts, allowing the scheduler to submit tasks in a round-robin manner.

Since SYCL is a higher-level C++ abstraction, the resource management is encapsulated in a dedicated struct named `SYCLStream` (see Listing 4.18).

```
#define N_STREAMS 2

using namespace std;

struct SYCLStream {
    sycl::queue q;
    float *d_A_f, *d_B_f, *d_C_f;
    sycl::half *d_A_h, *d_B_h, *d_C_h;

    SYCLStream(size_t max_elements) :
                q(sycl::gpu_selector_v,
                sycl::property::queue::in_order()) {
        d_A_f = sycl::malloc_device<float>(max_elements, q);
        d_B_f = sycl::malloc_device<float>(max_elements, q);
        d_C_f = sycl::malloc_device<float>(max_elements, q);

        d_A_h = sycl::malloc_device<sycl::half>(max_elements, q);
```

```
        d_B_h = sycl::malloc_device<sycl::half>(max_elements, q);
        d_C_h = sycl::malloc_device<sycl::half>(max_elements, q);

        if (q.get_device().get_info<sycl::info::device::name>()
                != "Intel(R) Arc(TM) Graphics") {
            std::cout << "Intel GPU NOT PRESENT\n";
            exit(EXIT_FAILURE);
        }
    }

    ~SYCLStream() {
        sycl::free(d_A_f, q); sycl::free(d_B_f, q); sycl::free(d_C_f, q);
        sycl::free(d_A_h, q); sycl::free(d_B_h, q); sycl::free(d_C_h, q);
    }
};

//static unique_ptr<sycl::queue> global_q;
static std::vector<std::unique_ptr<SYCLStream>> streams;
static int current_stream = 0;
static size_t MAX = (size_t)4096 * 4096;
```

Listing 4.18: sycl runner.cpp global data

The `SYCLStream` struct acts as a container that holds the state necessary for execution.

First, it contains a `sycl::queue`, which is the object used to submit commands to the device. The queue then is initialized with `in_order`, which simplifies the execution by ensuring that commands submitted to this queue are executed sequentially.

In the constructor, the device memory is pre-allocated using `sycl::malloc_device`, following the same optimization strategy used in the CUDA backend to avoid runtime allocation overhead. A specific check is also performed to verify that the code is running on the Intel GPU specifically, if the GPU is not found, the program aborts.

Finally, the struct includes a destructor, so that when the stream object is destroyed, all the memory buffers associated with it are explicitly freed, preventing memory leaks.

The execution logic is implemented in the following functions:

```
void sycl_gemm_32bit_p(float *A, float *B, float *C, int M, int N, int K) {
    SYCLStream& s = *streams[current_stream];

    s.q.memcpy(s.d_A_f, A, M * K * sizeof(float));
    s.q.memcpy(s.d_B_f, B, K * N * sizeof(float));

    float alpha = 1.0f, beta = 0.0f;

    oneapi::mkl::blas::row_major::gemm(s.q,
                                       oneapi::mkl::transpose::nontrans,
                                       oneapi::mkl::transpose::nontrans,
                                       M, N, K, alpha,
                                       s.d_A_f, K, s.d_B_f, N,
                                       beta, s.d_C_f, N);
```

```
15
16      s.q.memcpy(C, s.d_C_f, M * N * sizeof(float));
17      s.q.wait();
18
19      current_stream = (current_stream + 1) % N_STREAMS;
20  }
21
22  void sycl_gemm_16bit_p(sycl::half *A, sycl::half *B, sycl::half *C, int M,
        int N, int K) {
23      SYCLStream& s = *streams[current_stream];
24
25      s.q.memcpy(s.d_A_h, A, M * K * sizeof(sycl::half));
26      s.q.memcpy(s.d_B_h, B, K * N * sizeof(sycl::half));
27
28      sycl::half alpha = 1.0f, beta = 0.0f;
29      oneapi::mkl::blas::row_major::gemm(s.q,
30                                         oneapi::mkl::transpose::nontrans,
31                                         oneapi::mkl::transpose::nontrans,
32                                         M, N, K, alpha,
33                                         s.d_A_h, K, s.d_B_h, N,
34                                         beta, s.d_C_h, N);
35
36      s.q.memcpy(C, s.d_C_h, M * N * sizeof(sycl::half));
37      s.q.wait();
38
39      current_stream = (current_stream + 1) % N_STREAMS;
40  }
```

Listing 4.19: sycl runner.cpp execution

The `sycl_gemm_32bit_p` function (see Listing 4.19) orchestrates execution of the tasks, first, it selects the current stream and copies the input data from the host to the pre-allocated device buffers using `memcpy`.

Next, it invokes the matrix multiplication using the oneMKL library, thanks to this library the matrices A and B in this case can be passed directly with the `nontrans` flag, without needing to swap operands.

Finally, the result is copied back to the host, and `s.q.wait()` is called to block the execution until all operations in the queue are complete.

The 16-bit implementation is structurally identical but operates on the `sycl::half` data type. This allows the hardware to utilize half-precision units on the GPU, improving throughput.

```
1  extern "C" {
2      void sycl_init() {
3          init();
4      }
5
6      void sycl_gemm_32bit(void *A, void *B, void *C, int M, int N, int K) {
7          sycl_gemm_32bit_p((float*)A, (float*)B, (float*)C, M, N, K);
8      }
9
10     void sycl_gemm_16bit(void *A, void *B, void *C, int M, int N, int K) {
11         sycl_gemm_16bit_p((sycl::half*)A, (sycl::half*)B, (sycl::half*)C,
```

```
12                                  M, N, K);
13     }
14
15     void sycl_free() {
16         streams.clear();
17     }
18 }
```

Listing 4.20: sycl runner.cpp wrapper

The code concludes with the C-compatible wrappers. These functions simply cast the generic `void*` pointers to the appropriate C++ types (float* or sycl::half*) and forward the call.

The `sycl_free` function simply calls `streams.clear()`. Because the system uses smart pointers (std::unique_ptr) and the SYCLStream struct has a defined destructor, clearing the vector automatically triggers the destruction of all stream objects.

4.2.4 OpenBLAS

Finally, the project includes a backend to target the host CPU. Unlike the other accelerators which required complex compilation and separate dynamic libraries to ensure compatibility, the CPU implementation was introduced directly into the scheduler codebase.

As already said in this thesis, this library was the last addition to the project, and since the scheduler and the OpenBLAS library share the same compiler, there were no compatibility issues to solve, infact, the implementation is located directly in the header file `cpu.hpp` inside the `scheduler/` directory.

The complete implementation is presented below:

```
1  inline void cpu_init() {
2      openblas_set_num_threads(N_CORES);
3      omp_set_num_threads(N_CORES);
4  }
5
6  inline void cpu_gemm_32bit(void* A, void* B, void* C,
7                             int M, int N, int K) {
8
9      float* A_ = (float*) A;
10     float* B_ = (float*) B;
11     float* C_ = (float*) C;
12
13     cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
14                 M, N, K,
15                 1.0f,
16                 A_, K,
17                 B_, N,
18                 0.0f,
19                 C_, N);
20 }
```

```
21
22 inline void cpu_gemm_16bit(void* A, void* B, void* C,
23                             int M, int N, int K) {
24
25     uint16_t* pA = (uint16_t*)A;
26     uint16_t* pB = (uint16_t*)B;
27     uint16_t* pC = (uint16_t*)C;
28
29     std::vector<float> A_f32(M*K);
30     std::vector<float> B_f32(K*N);
31     std::vector<float> C_f32(M*N);
32
33     /* large overhead but we need it, not all cpus have 16 bit */
34     #pragma omp parallel for
35     for (int i = 0; i < M * K; ++i) {
36         A_f32[i] = half_to_float(pA[i]);
37     }
38
39     for (int i = 0; i < K * N; ++i) {
40         B_f32[i] = half_to_float(pB[i]);
41     }
42
43     cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
44                 M, N, K,
45                 1.0f,
46                 A_f32.data(), K,
47                 B_f32.data(), N,
48                 0.0f,
49                 C_f32.data(), N);
50
51     #pragma omp parallel for
52     for (int i = 0; i < M * N; ++i) {
53         pC[i] = float_to_half(C_f32[i]);
54     }
55 }
```

Listing 4.21: OpenBLAS CPU Implementation

The initialization function, `cpu_init`, is straightforward, it set the number of thread to use using `openblas_set_num_threads` and for OpenMP `omp_set_num_threads`. This ensures that the CPU utilizes all the available physical cores defined by the `N_CORES` macro.

For the 32 bit execution, the function `cpu_gemm_32bit` acts as a direct wrapper around the standard BLAS routine `cblas_sgemm`. It casts the generic pointers to `float*` and invokes the library function `CblasRowMajor`, which handles the input layout without transposition.

The 16-bit implementation (cpu_gemm_16bit), however, requires a workaround. Since the OpenBLAS library does not support half-precision arithmetic, the function performs a manual conversion. The logic is simple: first convert to Float,

1. Convert to float: the input 16 bit matrices are converted to standard 32 bit floats.

2. Compute: matrix multiplication is performed using the already available 32-bit routine.

3. Convert to half: The result is converted back to 16 bit.

To mitigate the performance cost of these conversions, the loops are parallelized using OpenMP (`#pragma omp parallel for`).

To handle the bitwise conversion between 16 bit and 32 bit formats without introducing dependencies, two standard IEEE 754 compliant functions were included in the project, these are well-known algorithms already widely used in the open-source community:

```
inline float half_to_float(uint16_t h) {
    uint32_t s = (h >> 15) & 0x00000001;
    uint32_t e = (h >> 10) & 0x0000001f;
    uint32_t m = h & 0x000003ff;

    if (e == 0) {
        if (m == 0) {
            uint32_t res = s << 31;
            float f; memcpy(&f, &res, 4); return f;
        } else {
            while (!(m & 0x00000400)) { m <<= 1; e -= 1; }
            e += 1; m &= ~0x00000400;
        }
    } else if (e == 31) {
        if (m == 0) { // Inf
             uint32_t res = (s << 31) | 0x7f800000;
             float f; memcpy(&f, &res, 4); return f;
        } else { // NaN
             uint32_t res = (s << 31) | 0x7f800000 | (m << 13);
             float f; memcpy(&f, &res, 4); return f;
        }
    }

    e = e + (127 - 15);
    m = m << 13;
    uint32_t res = (s << 31) | (e << 23) | m;
    float f; memcpy(&f, &res, 4); return f;
}
```

Listing 4.22: IEEE 754 Bitwise Conversion Helpers

The `float_to_half` function (available in the Appendix **??**) performs the inverse operation.

4.3 AMM Scheduler

The core of this project is the `AMScheduler` class. While the underlying system involves thread management, synchronization, and hardware interactions, the design goal was to encapsulate all this complexity behind a simple interface.

The scheduler is implemented as a C++ class defined in `scheduler.hpp`, is implemented as a **RAII** pattern. This ensures that the lifecycle of the execution environment including worker threads, shared buffers, and hardware contexts is tied to the lifecycle of the scheduler object itself. However, it is important to point out that while standard C++ smart pointers are used for the threads and maps, explicit memory management is maintained for performance critical data structures, such as the task arrays and the internal shared buffers allowing the system to minimize overhead and ensuring that the developer has full visibility into the exact behavior of the memory.

The public interface is minimal, consisting of a constructor, a destructor, and two primary methods for task submission and synchronization (see Listing 4.23):

```
class AMScheduler {

public:

    /* Constructor: initializes threads, benchmarks if needed and
    performancemaps */
    AMScheduler(Logic logic) {
        init_threads();
        init_benchmarks();
        init_maps();
    }

    /* Destructor: stops threads and cleans up resources */
    ~AMScheduler() {
        stop_threads();
    }

    /* Non blocking: pushes tasks to the coordinator */
    void do_tasks(task* tasks, size_t n) {
        pending_tasks += n;
        for (int i=0; i<n; i++) {
            tasks[i].start_time = chrono::high_resolution_clock::now();
            shared_buffers[BT::CORDINATOR]->put(&tasks[i]);
        }
    }

    /* Blocking: waits until all pending tasks are completed */
    void wait();

    /* Prints execution statistics */
    void print_stats(task* tasks, int num_tasks);
};
```

Listing 4.23: Scheduler Public Interface

The constructor takes a `Logic` parameter, which decide the scheduling strategy to be used. Upon instantiation, the scheduler perform three operations: it initializes the worker threads and their contexts, loads the benchmark data if presents, and prepares the performance maps. The destructor ensures a clean shutdown of the system, it invokes the `stop_threads` function, which signals all active threads to terminate, waits for them to join, and all all the backends clean functins.

The `do_tasks` method serves as the entry point for workload. Before pushing the task into the system, it records the current timestamp into each task. This timestamp is crucial for the profiler to accurately measure the end-to-end latency of each task. Once timestamped, the task pointer is pushed into the coordinator's shared buffer.

The Listing 4.24 is an example of how the scheduler is utilized in the main.cpp file:

```
void test_scheduler() {

    /* 1. Instantiate Scheduler (starts threads) */
    AMScheduler scheduler(Logic::DYNAMIC);

    /* 2. Submit work */
    scheduler.do_tasks(Array_Of_Input_Tasks, Number_of_input_tasks);

    /* ..... */

    /* 3. Wait for completion */
    scheduler.wait();

    /* 4. Analyze profiler results */
    scheduler.print_stats(Array_Of_Input_Tasks, Number_of_input_tasks);
}
```

Listing 4.24: Example usage

The usage of the scheduler is extremely simple, the developer only needs to instantiate the class passing the logic and submit the array of tasks. The scheduler handles all the complexity of thread management and hardware dispatching internally. For the explaination of the tasks creation see Sec. 4.3.4

4.3.1 Thread Initialization

The `init_threads` function is responsible for setting up the environment based on the available hardware (see Listing 4.25):

```
1  void init_threads() {
2
3      /* from the fastest to the slowest */
4      /* init here and not in the threads */
5  #ifdef ENABLE_CUDA
6          init_acc(BT::CUDA);
7          bts.push_back(BT::CUDA);
8  #endif
9  #ifdef ENABLE_SYCL
10             init_acc(BT::SYCL);
11             bts.push_back(BT::SYCL);
12 #endif
13 #ifdef ENABLE_OPENVINO
14             init_acc(BT::OPENVINO);
15             bts.push_back(BT::OPENVINO);
16 #endif
17 #ifdef ENABLE_OPENBLAS
18             init_acc(BT::OPENBLAS);
19             bts.push_back(BT::OPENBLAS);
20 #endif
21
22     /* initialize atomics for dynamic strategy */
23     if(strategy == Logic::DYNAMIC) {
24         #ifdef SLEEP
25         cout("[SCHEDULER] ERROR remove "#define SLEEP" with Dynamic logic")
26         exit(EXIT_FAILURE);
27         #endif
28         for (int i=0; i<bts.size(); i++) {
29             busy[bts[i]] = new atomic<bool>;
30             *busy[bts[i]] = false;
31         }
32     }
33
34     for (BT i : bts) {
35         shared_buffers[i] = make_unique<SharedBuffer>(BUFFER_LENGHT);
36         threads[i] = make_unique<thread>(&AMScheduler::worker_thread, this,
37                                          i, shared_buffers[i].get());
38     }
39
40     if (bts.size() == 0) {
41         PRINT("[SCHEDULER] ATTENTION, no accelerator\n");
42         exit(EXIT_FAILURE);
43     }
44
45     /* coordinator thread */
46     shared_buffers[BT::CORDINATOR] =
47                         make_unique<SharedBuffer>(BUFFER_LENGHT);
48
49     /* ref because the thread function try to copy all the parameters */
50     threads[BT::CORDINATOR] = make_unique<thread>(
51                                 &AMScheduler::coordinator_thread,
52                                 this, ref(shared_buffers));
```

```
53 }
```

Listing 4.25: Init Thread

This function first checks which accelerators were enabled during compilation using preprocessor macros passed with CMake. For each enabled backend, it calls the specific initialization function for each backend, inside the dynamic libraries, and adds the backend type to the active list `bts`. After it iterates through the active backends to create the worker threads. For each accelerator, a dedicated `SharedBuffer` is instantiated to hold its incoming tasks, and a standard C++ `std::thread` is spawned to run the `worker_thread` function.

Finally, the function initializes the **Coordinator Thread**. This thread is equipped with its own input buffer and is responsible for fetching tasks from the user and dispatching them to the workers (for implementation see Sec. 4.3.8). The reference to the shared buffers map is passed to the coordinator to allow it to push tasks to any worker inside the logics.

4.3.2 Workers

While the Coordinator thread manages the complexity of deciding where to send the data, the worker threads are designed to be extremely lightweight and simple. Their only purpose is to retrieve a task from their buffer, execute it, and signal the completion.

The implementation of the worker function (see Sec. 4.26):

```
1  void worker_thread(BT bt, SharedBuffer *buffer) {
2      task* task = nullptr;
3      PROF(profiler.init_last_work(bt));
4
5      if (strategy == Logic::DYNAMIC) {
6
7          /* Dynamic execution */
8          while (threads_keep_running) {
9
10             task = buffer->get();
11             if (!task) {continue;}
12
13             PROF(profiler.start_worker(bt));
14             handle_task(bt, task);
15             PROF(profiler.end_worker(bt));
16
17             *busy[bt] = false;
18             pending_tasks--;
19         }
20
21     } else {
22
23         /* Static execution */
24         while (threads_keep_running) {
```

```
25
26              task = buffer->get();
27
28              /* if there isn't a task we check if we have to shutdown */
29              if (!task) {continue;}
30
31              PROF(profiler.start_worker(bt));
32              handle_task(bt, task);
33              PROF(profiler.end_worker(bt));
34              pending_tasks--;
35          }
36
37      }
38  }
```

Listing 4.26: Worker Thread Implementation

The implementation is a continuous loop that retrieves a task from the shared buffer and processes it. The function `buffer->get()` is the entry point. As detailed in the Shared Buffer section (see Section 4.3.5), if the macro `SLEEP` is defined, this function blocks the thread and puts it to sleep until a new task arrives. This prevents the worker from consuming CPU cycles while waiting. If `SLEEP` is not defined, the thread spins (busy waiting) to react faster.

The behavior is slightly different based on the scheduler strategy:

- **Static:** The worker simply processes its queue as fast as possible without signaling.
- **Dynamic:** The worker notify the Coordinator when it finishes a task. It sets the atomic flag `*busy[bt]` to false so the Coordinator knows this accelerator is ready for a new task.

The `handle_task` function acts as a wrapper, isolating the scheduler from the specific backend implementations offered by the dynamic libraries linked at runtime.

```
1  /* task handler for the workers, choose the right backend_type */
2  inline void handle_task(BT backend_type, task *task) {
3      switch (backend_type) {
4      case BT::CUDA:
5  #ifdef ENABLE_CUDA
6          if (task->type == Type::FLOAT)
7              cuda_gemm_32bit(task->A,task->B,task->C,
8                              task->M, task->N, task->K);
9          if (task->type == Type::HALF)
10             cuda_gemm_16bit(task->A,task->B,task->C,
11                             task->M, task->N, task->K);
12 #endif
13         break;
14
15     case BT::SYCL:
16 #ifdef ENABLE_SYCL
17         if (task->type == Type::FLOAT)
```

```
18              sycl_gemm_32bit(task->A,task->B,task->C,
19                              task->M, task->N, task->K);
20          if (task->type == Type::HALF)
21              sycl_gemm_16bit(task->A,task->B,task->C,
22                              task->M, task->N, task->K);
23  #endif
24          break;
25
26      /* .. same for OpenVINO and OpenBLASS .. */
27
28      default:
29          cerr << "[SCHEDULER] ERROR handle task\n";
30          exit(EXIT_FAILURE);
31      }
32
33      task->end_time = chrono::high_resolution_clock::now();
34  }
```

Listing 4.27: Task Handler

This function (see Sec. 4.27) performs three simple steps, it checks the `backend_type` enum to decide which accelerator to use, then checks if the task requires 32-bit or 16-bit precision and calls the corresponding C-wrapper function from the dynamic libraries.Immediately after the computation finishes, it saves the current time into `task->end_time`, this gives the precise execution time for the metrics.

4.3.3 Benchmarks Routines

Once the threads are running, the coordinator thread needs specific data to make decisions inside the static logics (see Sec. 4.3.9). This data comes from the data profiling saved in CSV files create by `init_benchmarks` function and later uploaded inside the `PerformanceMap` structure (see Sec. 4.3.6) by the `init_maps` function.

```
1  /* upload the banchmark files and generate them if not presents */
2  void init_benchmarks() {
3      for (BT i : bts) {
4
5          string filename_f;
6          string filename_h;
7
8          filename_f = get_benchmark_filename(i, Type::FLOAT);
9          filename_h = get_benchmark_filename(i, Type::HALF);
10
11         if (filesystem::exists(filename_f))
12             PRINT("[SCHEDULER] File: " <<
13                     filename_f <<  " opened.\n");
14         else
15             benchmark(i, Type::FLOAT ,filename_f);
16
17         if (filesystem::exists(filename_h))
18             PRINT("[SCHEDULER] File: " <<
19                     filename_h <<  " opened.\n");
20         else
```

```
21              benchmark(i, Type::HALF ,filename_h);
22
23      }
24  }
25
26  /* create the map for each accelerator */
27  void init_maps() {
28      for (BT i : bts) {
29          string filename_f;
30          string filename_h;
31          filename_f = get_benchmark_filename(i, Type::FLOAT);
32          filename_h = get_benchmark_filename(i, Type::HALF);
33
34          if (filesystem::exists(filename_f)
35              && filesystem::exists(filename_h)) {
36
37              /* performance map creation */
38              bts_map[i] = make_unique<PerformanceMap>(STEP_SIZE,
39                                                       i, filename_f, filename_h);
40
41          } else
42              /* ... error prints and exit ... */
43      }
44  }
```

Listing 4.28: Init Benchmarks and Maps

The `init_benchmarks` function ensures that the data exists. It iterates through the list of active accelerators and checks for the existence of specific CSV files for both 32 and 16 bit data types. If a file is found, the system proceeds, if a file is missing, the system automatically triggers the benchmarking routine to generate the data from scratch.

Subsequently, again within the constructor, the `init_maps` function is called. Its role is to load the data from the disk into the memory. It reads the CSV files generated in the previous step and populates the `PerformanceMap` objects for each accelerator. This ensures that when the scheduler is running, it can query the expected execution time of a matrix multiplication in constant time O(1) without needing to read from the disk again. If a file is missing at this stage, the system considers it a critical error and shuts down, as the scheduler cannot function without its performance models. If a file is missing at this stage, the system shuts down, as the scheduler cannot function without its performance models.

The accuracy of the scheduler depends entirely on the quality of the data collected during the benchmarking phase (and small tweaks inside performanceMaps). This process is managed by the `benchmark` and `benchmark_acc` functions.

```
1  /* call benchmark_acc for each test matrix*/
2  inline void benchmark(BT bt, Type type, string filename) {
3      vector<int> dims;
4
5      for (int d = STEP_SIZE; d <= STEP_SIZE*STEP_TOTAL; d += STEP_SIZE)
6          dims.push_back(d);
```

```
7
8       for (int m : dims)
9           for (int n : dims)
10              for (int k : dims)
11                  benchmark_acc(bt, type, filename, m, n, k);
12
13      PRINT("[SCHEDULER] Benchmark ===\n");
14      PRINT("Results saved in bin/csv/" << filename << "\n");
15  }
```

Listing 4.29: Benchmark Function

The `benchmark` (see Listing 4.29) function acts as the driver. It defines the search space for the matrix dimensions (M, N, K) based on a `STEP_SIZE` (defined in the file `config.hpp`). It uses three nested loops to iterate through all possible combinations of dimensions, calling the measurement function for each configuration.

The measurement logic is implemented in `benchmark_acc` (see Listing 4.30):

```
1  /* benchmark the accelerator and write in the csv*/
2  inline void benchmark_acc(BT bt, Type type, string filename, int M, int N,
       int K) {
3      chrono::high_resolution_clock::time_point start_time;
4      chrono::high_resolution_clock::time_point end_time;
5      chrono::duration<double, std::milli> duration;
6
7      const int N_RUNS = 4;
8      const int N_TASKS = N_RUNS + 2;
9
10     double total_ms = 0.0;
11     double jit_ms = 0.0;
12     double no_jit_ms = 0.0;
13
14     /* tasks init */
15     task* tasks = init_tasks(N_TASKS, M, N, K, type);
16
17     /* warmup and jit detection */
18     start_time = chrono::high_resolution_clock::now();
19     handle_task(bt, &tasks[0]);
20     end_time = chrono::high_resolution_clock::now();
21     duration = end_time - start_time;
22     jit_ms = duration.count();
23
24     start_time = chrono::high_resolution_clock::now();
25     handle_task(bt, &tasks[1]);
26     end_time = chrono::high_resolution_clock::now();
27     duration = end_time - start_time;
28     jit_ms = jit_ms - duration.count();
29
30     /* runs */
31     for (int i = 2; i < N_TASKS; i++) {
32         start_time = chrono::high_resolution_clock::now();
33         handle_task(bt, &tasks[i]);
34         end_time = chrono::high_resolution_clock::now();
35         duration = end_time - start_time;
36         total_ms += duration.count();
37     }
```

```
38
39      clean_tasks(tasks, N_TASKS);
40
41      double avg_ms = total_ms/N_RUNS;
42
43      ofstream file(filename, ios::app);
44
45      if (file.is_open()) {
46          if (filesystem::file_size(filename) == 0)
47              file << "Accelerator,DataType,M,N,K,Avg_Time_ms,Jit_Time_ms\n";
48
49          string acc_str = get_acc_string(bt);
50
51          file << acc_str << "," << type << "," << M << "," << N <<
52                  "," << K << "," << avg_ms << "," << jit_ms << "\n";
53
54          file.close();
55      } else {
56          cerr << "[SCEDULER] ERROR Cannot open file: " << filename << "\n";
57      }
58  }
```

Listing 4.30: Benchmark Accelerator Function

Measuring the performance of accelerators like GPUs or NPUs is not trivial because these frameworks often perform a just in time compilation (JIT). This means that the very first time a specific matrix operation is executed, the driver must compile the kernel for the hardware, resulting in a significantly higher execution time compared to subsequent runs.

To handle this and isolate the overhead, the function uses a specific strategy; it allocates an array of tasks instead of reusing a single task object. This is done to prevent unrealistic CPU cache hits that might occur if the exact same memory pointers were reused constantly.

The measurement proceeds in three steps: the first task is executed. This run includes the time required for data transfer, computation, and any kernel compilation or driver initialization overhead. The second task is executed immediately after. Since the kernel is already compiled, this run represents the execution time without the JIT expenses, then the system calculates the **JIT time** by subtracting the duration of the second run from the duration of the first run.

The remaining tasks are executed in a loop, their durations are summed and divided by the number of runs to obtain a stable average execution time (`Avg_Time_ms`).

Finally, the results including the accelerator name, data type, matrix dimensions, average time, and JIT time are appended to the corresponding CSV file. This distinction between JIT time and execution time is crucial for the scheduler: during the first few iterations of an algorithm, the scheduler must account for the JIT overhead, and all of this complexity is hidden in the `PerformanceMap`.

4.3.4 Tasks

To allow the scheduler to manage workloads efficiently without being tied to a specific data type, the project introduces an abstraction called `Task`.

The definition of the task structure is below:

```
enum Type : int { FLOAT, HALF, UINT8, };

typedef struct {

    void *A;
    void *B;
    void *C;

    Type type; /* 1 float, 2 half, 3 8bit */
    int M;
    int N;
    int K;

    /* benchmarking */
    chrono::high_resolution_clock::time_point start_time;
    chrono::high_resolution_clock::time_point end_time;

} task;
```

Listing 4.31: Task Structure

The structure utilizes generic pointers (void *) for the matrices A, B, and C so it allows the same code to handle all the data type supported uniformly. The specific type of the data is determined by the Type enum field.

Furthermore, the task structure plays a crucial role in profiling, containing two high-resolution timestamp fields, `start_time` and `end_time`, the scheduler can records the exact moment a task is submitted, and the worker thread records the moment the computation finishes. This allows for precise calculation of the latency and throughput.

The typical lifecycle of a task within the application is simple:

1. The tasks are allocated and filled with data in the main thread.
2. They are passed to the scheduler via `do_tasks` function exposed via the scheduler object.
3. Once processing is complete, the memory is explicitly freed with the `clean_tasks` function.

To automate the data generation process for testing every single workload supported by the scheduler, the `init_tasks` function was used to create arrays of identical matrices, while `init_hetero_tasks` was employed to create arrays of heterogeneous

matrices. These datasets constitute the input used for all the tests performed on the scheduler.

```
inline task* init_tasks(size_t n_task, int m, int n, int k, Type t) {
    task* array_task = new task[n_task];

    for (int i = 0; i < n_task; i++) {
        array_task[i].M = m;
        array_task[i].N = n;
        array_task[i].K = k;
        array_task[i].type = t;

        if (t == Type::FLOAT) {
            array_task[i].A = new float[m * k];
            array_task[i].B = new float[k * n];
            array_task[i].C = new float[m * n];

            init_32bit(array_task[i].A,array_task[i].B,array_task[i].C, m,
    n, k);

        } else if (t == Type::HALF) {

            /* .. same as 32 bit .. */
        }
    }

    return array_task;
}
```

Listing 4.32: Init Tasks Function

The implementation is straightforward, the function `init_task` simply take the number of tasks to create, the matrix dimensions and the desired type of the matrices, it allocate the array of tasks, then the system populate the matrices with random numbers thanks to the `init_32bit` function:

```
inline void init_32bit(void* A, void* B, void* C, int M, int N, int K) {
    float* pA = (float*)A;
    float* pB = (float*)B;
    float* pC = (float*)C;

    random_device rd;
    mt19937 gen(rd());

    uniform_real_distribution<float> dis(-1.0f, 1.0f);

    for (int i = 0; i < M * K; ++i)
        pA[i] = dis(gen);

    for (int i = 0; i < K * N; ++i)
        pB[i] = dis(gen);

    for (int i = 0; i < M * N; ++i)
        pC[i] = 0.0f;
```

```
19 }
```

Listing 4.33: Init 32 bit Function

The `init_32bit` function casts the generic void pointers back to float* and uses the C++ standard random number generator to fill the input matrices A and B. The output matrix C is initialized to zero for the results.

The 16 bit initialization function is omitted because it follows the same pattern but includes an additional step to convert the generated random floats into half precision bit, same for the function `init_hetero_tasks`, it simply iterate on random number to create the a stream of random dimension matrices.

4.3.5 SharedBuffer

The SharedBuffer is the tool used for communication between the threads, since the Coordinator and the Workers run independently, they need a safe place to exchange data.

```
class SharedBuffer {
    private:
        unique_ptr<task*[]> data;
        size_t size = 0;

        alignas(64) std::atomic<size_t> write_i{0};
        alignas(64) std::atomic<size_t> read_i{0};

    public:
        /* constructor */
        SharedBuffer(size_t s) : size(s), data(new task*[s]) {}

        ~SharedBuffer() {}

        /* put one task pointer inside, sleep if full*/
        void put(task* ele) {

#ifdef SLEEP
            while (((write_i + 1) % size) == read_i) {
                usleep(PUT_SLEEP);
            }
#else
            while (((write_i + 1) % size) == read_i) {}
#endif
            data[write_i] = ele;
            write_i = (write_i + 1) % size;
        }

        /* get one task pointer sleep if empty, return to the caller after
         * 5 attempt*/
        task* get() {
            int c = 0;

#ifdef SLEEP
            while (write_i == read_i){
                c++;
                usleep(GET_SLEEP);
                if (c >= 5) {return nullptr;}
            }
#else
            while (write_i == read_i){
                c++;
                if (c >= 50) {return nullptr;}
            }
#endif
            task* ele = data[read_i];
            read_i = (read_i + 1) % size;

            return ele;
        }
};
```

Listing 4.34: Shared Buffer Implementation

This class was implemented specifically for this scheduler to be as simple and fast as possible, avoiding the complex locking mechanisms usually found in standard libraries.

The logic is based on a circular buffer managed by two counters: `write_i` (where to put new data) and `read_i` (where to take data) and by using atomic variables, the threads can update these counters without interfering with each other.

The put function is used by the Coordinator, it checks if there is space, if the buffer is full, it waits. The get function is used by the Worker to pick up a task, if the buffer is empty, it waits for new data.

The task array is encapsulated within a `std::unique_ptr`, this ensures that the memory is automatically released when the associated SharedBuffer object is destroyed. However, the underlying structure remains a simple raw array.

Is important to note that the `alignas(64)` directive is applied to the index variables to prevent false sharing. This is a standard optimization technique, by forcing the read and write counters to reside on separate CPU cache lines (thanks to the padding that the compiler insert), it ensures that the Coordinator and Worker threads can update their respective counters independently without triggering unnecessary synchronization traffic between the processor cores, speeding up the process.

A key detail is in the get function, that it doesn't wait forever, as shown in the code, if the buffer remains empty for a certain number of attempts (5 with sleep, 50 without), the function returns nullptr. This is done for a specific reason, the worker threads run inside a while loop controlled by the scheduler. If the get function blocked indefinitely waiting for a task, the thread would get stuck there and could never check if the program is trying to shut down. By returning control to the caller periodically, the worker can check if it should stop running or keep waiting for new work.

By default, this is a busy waiting: the thread keeps checking the variable continuously, this is very fast but consumes all of the CPU core. To fix this, the `SLEEP` macro can be enabled. The thread takes a small sleep if it has to wait, freeing up the CPU. This small feature was implemented primarily for future optimizations for maximizing resource efficiency. However, since it inevitably introduces a slight overhead (increased latency), it remains disabled and was not utilized during the experimental benchmarks presented in this thesis.

4.3.6 Performance Map

The Performance Map is a specialized data structure utilized by the Coordinator thread. Its primary purpose is to act as a predictive model, providing the expected execution time for a specific task on a specific accelerator.

As explained in the previous section the raw data is acquired through automated benchmark routines (see Sec. 4.3.3). However, reading from CSV files isn't an option latency wise, therefore, the PerformanceMap class loads this data into an optimized in-memory structure at startup.

The class definition and its global variables are presented below:

```
class PerformanceMap {

    struct map_struct {
        unordered_map<unsigned long long, tuple<double, double>> grid_map;

        long long last_M = -1;
        long long last_N = -1;
        long long last_K = -1;
        double last_result = -1.0;
        double max_result = -1.0;
    };

    map_struct map_f;
    map_struct map_h;

    unordered_set<unsigned long long> jit_cache_f;
    unordered_set<unsigned long long> jit_cache_h;

    /* which accelerator am i */
    BT bt;

    long long STEP_SIZE;

public:

    PerformanceMap(int step_size, BT bt,
                   string filename_f,
                   string filename_h)
                   : STEP_SIZE(step_size), bt(bt) {
        init_maps(filename_f, filename_h);
    }

    double query(long long M, long long N, long long K, Type type, bool
    add_jit);
```

The core of the structure is the `map_struct`, which contains an `unordered_map`. This map uses an unsigned long long as a key (explained shortly) and stores a tuple containing the execution time and the JIT compilation time. To avoid redundant lookups for homogeneous workloads (where tasks are identical), the structure also includes a caching mechanism, `last_M`, `last_N`, and `last_result` that returns the previous result immediately if the dimensions match.

Two separate instances of this struct are maintained: `map_f` for 32 bit floating point tasks and `map_h` for 16 bit half precision tasks, additionally, the `jit_cache` sets are used to track which matrix dimensions have already triggered a JIT compilation, allowing the system to predict when the "cold start" penalty must be paid. The bt variable identifies which accelerator this map belongs to, enabling specific heuristics for each hardware type.

The public interface is minimal: besides the constructor that loads the data from files, only the query function is exposed to the Coordinator.

To index the 3D matrix dimensions efficiently within a hash map, a bit-packing strategy was chosen over using complex objects or tuples as keys. This approach minimizes hashing overhead and ensures fast lookups.

```
/* return the successor of val in our step_size */
long long round_up(long long val) {
    if (val == 0) return STEP_SIZE;
    return ((val + STEP_SIZE - 1) / STEP_SIZE) * STEP_SIZE;
}

/* return the key of val in our step_size */
unsigned long long get_key(long long M, long long N, long long K) {
    long long rM = round_up(M);
    long long rN = round_up(N);
    long long rK = round_up(K);

    /* 64 bit  |4bit nil|20bit M|20bit N|20bit K| */
    return ((unsigned long long)rM << 40) | ((unsigned long long)rN << 20)
    | (unsigned long long)rK;
}
```

Listing 4.35: Get key Implementation

The `get_key` function first rounds up the dimensions to the nearest `STEP_SIZE` (the interval used during benchmarking) to ensure that arbitrary task sizes map to the nearest available data point. Then, it packs the three task dimensions into a single 64 bit integer by shifting bits: M the high 20 bits, N the middle, and K the low bits. This creates a unique integer identifier for any matrix configuration.

The query function is the main part of this component, it takes the matrix dimensions, the data type, and a boolean flag `add_jit`, part of the code is shown below:

```
/* return the time, add_jit true = populate the jit cache */
double query(long long M, long long N,
             long long K, Type type, bool add_jit) {

    /* ... setup pointers to the correct map and cache ... */

    unsigned long long key = get_key(M, N, K);

    /* we cache the last results */
    if (M == data->last_M && N == data->last_N && K == data->last_K)
        {return data->last_K;}

    data->last_M = M; data->last_N = N; data->last_K = K;

    /* return if we find the key */
    if (data->grid_map.find(key) != data->grid_map.end()) {
        time_ms = get<0>(data->grid_map[key]);
        jit_ms = get<1>(data->grid_map[key]);
        /* add jit_ms only if openvino never see M N K*/
        if (bt == BT::OPENVINO) {
            /* mitigate overhead when other accelerator are running */
            time_ms += time_ms * 0.1;
            if (jit_cache->find(key) == jit_cache->end()){
                if (add_jit)
                    jit_cache->insert(key);
                time_ms += jit_ms * 1.3;
            }
        }

        /* add jit_ms only if sycl never see N */
        if (bt == BT::SYCL) {
            /* mitigate overhead when other accelerator are running */
            time_ms += time_ms * 0.1;
            unsigned long long sycl_key = (unsigned long long) N;
            if (jit_cache->find(sycl_key) == jit_cache->end()){
                if (add_jit) jit_cache->insert(sycl_key);
                time_ms += JIT_MS_SYCL;
            }
        }

        /* mitigate openblas error in bencharks  */
        if (bt == BT::OPENBLAS) time_ms = time_ms * 0.5;
        data->last_K = time_ms;
        return time_ms;
    }

    /* if we don't have data on this matrix, just return max time * 2 */
    time_ms = data->max_result * 2;
    /* add jit expences */
    if (bt == BT::SYCL || bt == BT::OPENVINO) time_ms += JIT_MS_SYCL;
    /* cache the result */
    data->last_K = time_ms;
    return time_ms;
}
```

Listing 4.36: Query Implementation

The `add_jit` boolean serve a specific task, the Coordinator often needs to query the cost of a task purely for planning purposes, without actually scheduling it. In these cases, `add_jit` is set to false, preventing the map from marking the matrix as "seen" in the JIT cache, ensuring that the compilation cost is correctly applied only when the task is actually dispatched.

The function `query` implements specific heuristics derived and observed from extensive black box analysis of the drivers, as their internal implementations are not public:

- **CUDA**: For the nvidia backend, no JIT logic is applied, the driver handles kernel caching transparently and efficiently, so the raw execution time from the benchmark is returned directly.
- **OpenBLAS**: Similarly, the CPU backend has no compilation overhead, however, we observed that during the benchmark routines, the processor often failed to maintain the maximum Boost frequency consistently or suffered from thermal throttling, resulting in benchmark times that were slightly slower than real world execution. To correct this, a scaling factor is applied to the estimated time to align it with the actual performance observed during runtime.
- **OpenVINO**: The NPU driver behavior was straightforward, the JIT compilation times measured during benchmarking matched the runtime behavior perfectly. Therefore if a matrix key is not in the `jit_cache`, the stored JIT time is added to the result. Additionally, we observed that when the NPU runs in parallel with other accelerators, it suffers from a slight performance degradation due to system bus contention or power sharing. To mitigate this, a small percentage of overhead is added to the base execution time.
- **SYCL**: This was the most complex driver to analyze. Our tests revealed that the driver compiles an optimized kernel for a wide range of matrix sizes, but it triggers a recompilation whenever the N dimension changes. Furthermore, the compilation cost was found to be constant, around 100-120 ms, regardless of the matrix size. Consequently, the benchmarked JIT times were discarded in favor of adding a fixed constant `JIT_MS_SYCL`) whenever a new N is encountered. Similar to the NPU, the Intel GPU also exhibits slight overhead during parallel execution, which is compensated for by adding a small factor to the predicted time.

All of these tweaks exhibit, as it will be shown in the next section, a good behavior in predicting the time of completition of a task, furthermore the code used for the black box analysis is intirely in the tests.hpp file, found in Appendix **??**.

4.3.7 Profiler

To evaluate the performance and the efficiency of the scheduling logics, the `Profiler` tool was developed integrated directly into the scheduler's codebase. It acts as an observer, recording precise timestamps and sensor data during the execution of workloads without interfering with the logic.

The class provides a simple interface to start and stop counters for various metrics. Its primary responsibilities are:

- Coordinator Latency, tracking the exact time spent in specific phases Fetching, Logic, Dispatching using high-resolution clocks.
- Worker Statistics, monitoring each worker thread to calculate occupancy.
- Power Monitoring, measuring the total energy consumed by the hardware during the workspan.

See Listing 4.37 for a simplified view of the class structure and its key components:

```

/* Consumption stats */
struct EnergyConsumption {};
/* Coordinator stats */
struct Metric {};
/* Workers stats */
struct WokerMetric {};

class Profiler {
public:

/* power monitor helpers */
void start_power_monitor() {
    #ifdef ENABLE_INTEL_POWER_PROFILE
    /* read RAPL files */
    #endif

    #ifdef ENABLE_CUDA
    /* call NVML library */
    #endif
}
void stop_power_monitor();

/* scheduler logic monitor helpers */
void start_fetch();
void start_logic();
void start_dispatch();
void stop_fetch();
void stop_logic();
void stop_dispatch();
void init_last_work();
void start_worker();
void end_worker();
void record_sample();
```

```
35
36 void print_stats();
37 void save_to_csv();
38
39 };
```

Listing 4.37: Profiler Class Structure

For the NVIDIA GPU, collecting data was straightforward thanks to the NVML library, the function `nvmlDeviceGetTotalEnergyConsumption` provides a monotonic counter of the total energy consumed by the board in millijoules since the driver started. By reading this counter before and after the workload, the exact consumption is derived.

For the Intel hardware the system relies on RAPL. On Linux systems these hardware counters are exposed as standard text files in `/sys/class/powercap/`, the Profiler reads the energy values in microjoules directly from these files. While RAPL allows for fine grained domains (like DRAM or specific cores), monitoring the entire CPU Package was deemed sufficient for our goals, as we are interested in the total system efficiency gain, furthermore the GPU and NPU are also included in the CPU package.

Using Joules as the primary metric provides a way to compare efficiency, as it inherently accounts for both the power draw and the duration of the task.

Finally, all collected metrics are aggregated and appended to a results.csv file. This automated data collection was crucial for generating the results presented in the next chapter.

4.3.8 Coordinator

The heart of the Scheduler is the Coordinator thread. It is the decision making engine that determines how tasks are distributed among the available workers. As previously mentioned, the coordinator supports multiple task dispatch logics, ranging from simple baselines to more complex algorithms.

Just like the worker threads, the Coordinator operates within a continuous while loop controlled by the `threads_keep_running` atomic flag. Before the loop, a switch statement selects the active logic based on the configuration provided during the scheduler's object instantiation.

Baseline: CUDA only and Round Robin

To validate the internal architecture and establish a performance baseline, two simple strategies were implemented first.

`CODA_ONLY` is the most basic logic simply bypasses the heterogeneity of the system and dispatch all the tasks to the fastest accelerator. This serves as the primary baseline: any strategy must outperform this single device execution at least to be considered effective.

```
case Logic::CUDA_ONLY:

PROF(profiler.start_power_monitor());

if (buffers[BT::CUDA] == nullptr) {
    cerr << "[SCHEDULER] Error, cuda logic but no cuda card";
    exit(EXIT_FAILURE);
}

while (threads_keep_running) {
    single_task = buffer->get();
    if (!single_task) {continue;}
    buffers[BT::CUDA]->put(single_task);
}

PROF(profiler.stop_power_monitor());

break;
```

Listing 4.38: CUDA Only Logic

The code is straightforward, it fetches a task from the input buffer and immediately pushes it into the CUDA worker's buffer. The PROF macros are used to trigger the start and stop of the monitoring tools within the Profiler, and thanks to this macro, can be turned off for minimizing the overhead simply by not defining `ENABLE_PROFILING`.

To verify the system's ability to manage multiple accelerators concurrently, a Round

Robin logic was implemented.

```
case Logic::ROUND_ROBIN:

while (threads_keep_running) {
    single_task = buffer->get();
    if (!single_task) {continue;}

    buffers[bts[i]]->put(single_task);

    i = (i+1) % bts_len;
}
break;
```

Listing 4.39: Round Robin Logic

This logic distributes tasks equally among all available workers in a cyclic manner, while effective for validating, this strategy is definitely not performance oriented.

4.3.9 Static Partitioning

This strategy represents the core logic for handling homogeneous workloads (streams of identical matrices) in a batch style manner. Ideally, to minimize the makespan, the workload should be distributed proportionally to the computing power of each accelerator.

The algorithm operates in batches defined by `BATCH_SIZE`. The execution flow is divided into three distinct phases: Fetch, Logic, and Dispatch.

First, the coordinator retrieves a batch of tasks from the input buffer. Then, it queries the PerformanceMap to estimate the execution time for a single task on each accelerator.

```
case Logic::STATIC_PARTITIONING:

PROF(profiler.start_power_monitor());

while (threads_keep_running) {
    /* fetch */
    PROF(profiler.start_fetch());
    while (c < BATCH_SIZE) {
        tasks[c] = buffer->get();
        if (!tasks[c]) {break;}
        c++;
    }
    PROF(profiler.stop_fetch());

    if (c == 0) {continue;}

    /* calculate speeds */
    PROF(profiler.start_logic());
    total_speed = 0.0;
    for (int j=0; j<bts_len; j++){
```

```
21          double ms = bts_map[bts[j]]->query(tasks[0]->M, tasks[0]->N,
22                                             tasks[0]->K, tasks[0]->type,
    true);
23
24          /* speed = 1 / time */
25          acc_speed[j] = 1.0 / (ms + 1e-9);
26          total_speed += acc_speed[j];
27      }
28
29      /* .... */
```

Listing 4.40: Static Partitioning: Fetch and speed

The speed of each accelerator j is calculated as the inverse of its execution time:

$$v_j = 1/t_j$$

and it will be used for the task ratios for each accelerator.

As a standard practice, a small epsilon is added to avoid division by zero, in our logic its just 1 nanosecond, so it doesn't interfere with the times, but is useful in case of data errors from the query.

Once the total system speed (V) is known (simply the sums of each accelerator speed), the ideal share of tasks for each accelerator (S_j) is calculated as:

$$S_j = \frac{v_j}{V} \cdot N_{task}$$

Since tasks cannot be split, this floating-point share must be converted into an integer.

```
1  /* .... */
2
3  /* exact task per acc (double) */
4  double shares[bts_len];
5  int total_assigned = 0;
6
7  for (int j=0; j<bts_len; j++) {
8      double ratio = acc_speed[j] / total_speed;
9      shares[j] = ratio * (double)c;
10     n_acc_task[j] = (int)shares[j]; /* integer part */
11     total_assigned += n_acc_task[j];
12 }
13
14 int missing_c = c - total_assigned;
15
16 /* assign the missing task to the accelerator
17  * mitigate slowness between accelerators */
18 while (missing_c > 0) {
19     int best_j = -1;
20     double max_decimal = -1.0;
21
22     /* find the accelerator with the highest decimal part */
23     for (int j=0; j<bts_len; j++) {
```

```
24          double decimal_part = shares[j] - (double)n_acc_task[j];
25          if (decimal_part > max_decimal) {
26              max_decimal = decimal_part;
27              best_j = j;
28          }
29      }
30
31      /* assign one more task to the winner */
32      if (best_j != -1) {
33          n_acc_task[best_j]++;
34          /* set to -1 so it won't be picked again */
35          shares[best_j] = -1.0;
36          missing_c--;
37      } else {
38          /* fallback */
39          n_acc_task[0]++;
40          missing_c--;
41      }
42  }
43
44  prof(profiler.stop_logic());
45
46  /* .... */
```

Listing 4.41: Static Partitioning: Distribution Logic

The integer conversion inevitably leads to a truncation error (e.g., 3.7 tasks becomes 3), leaving some tasks unassigned, inside `missing_c`. To solve this problem and ensure complete fairness between tasks, the scheduler implements the **Largest Remainder Method** for the decimal part of the array `shares`.

Let's consider a simple example, a batch of 10 tasks distributed between two accelerators A and B, where the logic assigns: 4.7 tasks to A and 5.3 tasks to B. If we simply truncate integers A gets 4 tasks and B gets 5 tasks, so the total assigned will be 9 and we are missing 1 task.

To decide who receives this unassigned task, the scheduler looks at the decimal remainders. A lost 0.7 and B lost 0.3. Since $0.7 > 0.3$, the accelerator A has the "highest claim" to the extra task. Therefore, the final distribution becomes 5 for A and 5 for B, correctly summing to 10. This method ensures that the final integer distribution remains as close as possible to the ideal proportion calculated before.

Finally, the calculated number of tasks in `n_acc_task[j]` is dispatched to the corresponding worker buffers.

```
1
2       /* .... */
3
4       PROF(profiler.start_dispatch());
5
6       /* send task to the accelerators */
7       t = 0;
8       for (int j=0; j<bts_len; j++) {
9           for (int w=0; w<n_acc_task[j]; w++){
```

```
10              if (t >= c) break;
11
12              buffers[bts[j]]->put(tasks[t]);
13              t++;
14          }
15      }
16
17      c = 0;
18      PROF(profiler.stop_dispatch());
19      PROF(profiler.record_sample());
20  }
21
22  PROF(profiler.stop_power_monitor());
23  break;
```

Listing 4.42: Static Partitioning: Dispatch

4.3.10 Large Matrix Split

This static strategy addresses a specific case, handling a single matrix multiplication that is too large to fit into the memory of any individual accelerator. Instead of treating the matrix as an indivisible unit, this logic implements a Split-Compile-Merge approach, splitting the large matrix horizontally in smaller chunks and sending smaller tasks to each accellerator, enabling the sistem to process this workload in parallel, then the result are written directly into the corresponding section.

To determine the optimal number of rows for each device, the scheduler employs an **Iterative Refinement algorithm**. This logic is also splitted in 3 phase, the initial estimation, the iterative refinement and then the dispatchign.

```
case Logic::LARGE_MATRIX_SPLIT:
PROF(profiler.start_power_monitor());

while (threads_keep_running) {
    PROF(profiler.start_fetch());

    single_task = buffer->get();
    if (!single_task) {continue;}

    PROF(profiler.stop_fetch());
    PROF(profiler.start_logic());

    double curr_speeds[bts_len];
    int rows[bts_len];

    int rest = single_task->M % bts_len;
    int base = single_task->M / bts_len;

    /* equal row initially */
    for (int j=0; j<bts_len; j++)
        rows[j] = base + (j < rest ? 1 : 0);

    /* .... */
```

Listing 4.43: Split Logic: Initialization

The process begins by retrieving the large task and distributing the rows as evenly as possible among all available accelerators. This serves as a starting point for the refinement loop.

The core of the logic is a feedback loop that runs for a fixed number of iterations (decided with `SPLIT_MATRIX_ITERATION`). In each iteration, the scheduler queries the PerformanceMap to estimate how fast each accelerator can process its currently assigned chunk.

```
/* .... */
/* loop for adjusting the ratios */
for (int i = 0; i < SPLIT_MATRIX_ITERATION; i++) {
    double total_speed = 0.0;

    /* current speed for 1 row */
    for (int j=0; j<bts_len; j++) {
        int r = (rows[j] > 0) ? rows[j] : 1;

        double ms = 0.0;

        ms = bts_map[bts[j]]->query(r,
                    single_task->N, single_task->K, single_task->type,
    false);

        /* update time if r exceed max size */
        if (r > MAX_SIZE){
            double max_ms = bts_map[bts[j]]->query(MAX_SIZE,
                single_task->N, single_task->K, single_task->type, false);
            ms = max_ms * ((double)r / (double)MAX_SIZE);
        }

        /* single row speed, number of rows / ms */
        double speed = (double)r / (ms + 1e-9);

        curr_speeds[j] = speed;
        total_speed += speed;
    }

    /* adjust the row with the ratios */
    int rows_distributed = 0;
    for (int j=0; j<bts_len; j++) {
        double ratio = curr_speeds[j] / total_speed;

        /* new rows from the ratio */
        int target_rows = (int)(ratio * single_task->M);

        rows[j] = target_rows;
        rows_distributed += rows[j];
    }
}

PROF(profiler.stop_logic());
/* .... */
```

Listing 4.44: Split Logic: Refinement Loop

The metric used is rows per millisecond (rows / ms):

- The scheduler queries the map for the time required to compute the currently assigned number of rows.
- If the number of rows exceeds the maximum size present in the performanceMap, the time is estimated via $T_{new} = T_{max} \cdot \frac{row}{maxrow}$
- The throughput for each device is calculated, and the total rows M are redistributed proportionally to these new speeds.

This loop converge towards a balanced distribution where all accelerators finish at approximately the same time, and once the optimal row counts inside `rows[j]` are calculated, the actual tasks must be created and dispatched. This step does not copy any data, instead, it uses pointer arithmetic to create the smaller tasks.

```
/* .... */
    size_t elem_size = (single_task->type == Type::FLOAT) ? 4 : 2;
    size_t offset_rows_acc = 0;

    /* dispatch tasks */
    for (int j=0; j<bts_len; j++) {
        int h = rows[j];
        if (h <= 0) continue;

        /* divide rows into tasks if exceed MAX_SIZE */
        while (h > 0) {
            int h_limit = h;

            if (h_limit > MAX_SIZE) h_limit = MAX_SIZE;

            task* sub_task = new ::task;
            *sub_task = *single_task;

            /* save the pointer for later freeup */
            sub_task_array.push_back(sub_task);

            /* A row */
            sub_task->M = h_limit;

            /* A offset */
            size_t offset_A = offset_rows_acc * single_task->K * elem_size;
            sub_task->A = (char*)single_task->A + offset_A;

            /* B offset still all in memory */
            sub_task->B = single_task->B;

            /* C offset */
            size_t offset_C = offset_rows_acc * single_task->N * elem_size;
            sub_task->C = (char*)single_task->C + offset_C;

            /* add task to the pending */
            pending_tasks++;

            /* submit to the acc */
            buffers[bts[j]]->put(sub_task);

            /* for each accellerator we add more offset */
            offset_rows_acc += h_limit;

            h -= h_limit;
        }
    }

    pending_tasks--;
}
```

Listing 4.45: Split Logic: Sub Task dispatch

For each accelerator, the logic creates a new task structure:

1. **A**: The pointer is shifted forward to skip the rows processed by previous accelerators.
2. **B**: The pointer remains unchanged, as B is needed in its entirety for every row of A.
3. **C**: The pointer is shifted similarly to A, ensuring that the accelerator writes its result into the correct slice of the output buffer.

Furthermore, if the assigned chunk of rows is still too large for the device memory, larger than `MAX_SIZE`, the chunk is further subdivided into smaller, sequential tasks for that specific worker.

4.3.11 Static Hetero Partitioning

This strategy is an evolution of the Static Partitioning logic, designed for heterogeneous workloads where tasks differ in size and computational cost (stream of matrices with random dimensions).

In this scenario, calculating a simple ratio based on the first task (as done in the previous strategy) would be incorrect, because the speed of an accelerator is not constant but depends on the problem size. Therefore, instead of assigning tasks based on a global ratio, this algorithm adopts a **greedy approach**: it simulates the execution timeline and assigns each task to the accelerator that can complete it soonest, considering its current accumulated load.

```
/* ... fetch logic same as before ... */

/* iterator for each bt */
int i_bt[bts_len];

for (int i=0; i<bts_len; i++)
    i_bt[i] = 0;

task* dispatch_tasks[bts_len][BATCH_SIZE_HETERO];

/* for every task */
for (int i = 0; i < c; i++) {
    int best_bt = -1;
    double min_ms = 1e15;

    /* for every bt */
    for (int j = 0; j < bts_len; j++) {
        double query_ms = bts_map[bts[j]]->query(tasks_hetero[i]->M,
                tasks_hetero[i]->N, tasks_hetero[i]->K,
                tasks_hetero[i]->type, false);

        double finish_ms = bt_ms[j] + query_ms;

        /* we assign the task to the least "filled" */
        if (finish_ms < min_ms) {
            min_ms = finish_ms;
            best_bt = j;
        }
    }

    /* openvino and sycl jit updates */
    if (bts[best_bt] == BT::OPENVINO || bts[best_bt] == BT::SYCL)
        bts_map[bts[best_bt]]->query(tasks_hetero[i]->M,
            tasks_hetero[i]->N, tasks_hetero[i]->K,
            tasks_hetero[i]->type, true);

    dispatch_tasks[best_bt][i_bt[best_bt]] = tasks_hetero[i];
    i_bt[best_bt]++;
    bt_ms[best_bt] = min_ms;
}

/* .... */
```

Listing 4.46: Static Hetero Logic

The core relies on the `bt_ms` array, wich tracks the load assigned to each accellerator so far (in the current batch). The load stand for the accumulated ms expected for that particular accellerator.

The logic iterates through the batch of tasks one by one. For each task, it queries the PerformanceMap to predict how long it will take on every available accelerator, then it compute the finish time expected with the load plus the task expences, than it update the best `bt`. Finally the task then is assigned to the best accellerator that has the smallest finish time, and the current load of that accellerator is updated.

Additionally, once a decision is made, the query function is called again with `add_jit = true` for the chosen accelerator, ensures that the JIT cache is updated.

After the simulation loop processes the entire batch, the tasks are pushed to the worker buffers same as before.

4.3.12 Dynamic

The Dynamic strategy was the final logic implemented in the scheduler. Unlike the static approaches which rely on performance map and pre calculated distributions, this strategy implements a simple first come first served method. Since the scheduler's architecture was originally designed around batch processing and static partitioning, this logic was adapted to fit the existing structure by introducing a polling mechanism to monitor worker status (already seen in the worker section, see Sec. 4.3.2).

The Coordinator effectively polls the state of the worker threads to find one that is currently idle.

```
case Logic::DYNAMIC:
i = 0;
while (threads_keep_running) {

    if (dispached) {
        single_task = buffer->get();
        if (!single_task) {continue;}
        dispached = false;
    }

    if (*busy[bts[i]] == false) {
        *busy[bts[i]] = true;
        dispached = true;
        buffers[bts[i]]->put(single_task);
    }

    i++;
    i = i % bts_len;
}
```

Listing 4.47: Static Hetero Logic

The Coordinator fetches a new task from the input buffer only if the dispatched flag is true, ensures that the previous task was dispatched to some idle accelerator. The loop iterates through the list of accelerators `bts` and checks the busy atomic flag of the current worker. If `*busy[bts[i]]` is false (worker is idle), the task is immediately pushed to that worker's buffer. The busy flag is then set to true, and dispatched is set to true to trigger the fetch of the next task. If the current accelerator is busy, the loop increments i to check the next one in the list, ensuring that all workers are polled fairly in a round robin manner.

This approach minimizes the scheduler's complexity but introduces a busy wait cycle on the Coordinator as it constantly checks for free workers.

As shown later on in the results chapter, this method performs efficiently in homogeneous workloads, However, due to its greedy nature and lack of knowledge about task complexity or hardware capabilities, it tends to underperform in highly heterogeneous environments, where it fails to optimize load balancing effectively.

Chapter 5

Results

This chapter, discusses the performance of the AMM Scheduler. The objective of the discussed tests is the validation of the effectiveness of the proposed scheduling logics (Static, Static Hetero, and Dynamic) and to analyze how the workload is distributed among the available accelerators in a real-world heterogeneous environment.

All the experiments were conducted as already described in the Tool chapter (see Chap. 2), on a modern high performance laptop, specifically the Lenovo Yoga Pro 9.

The specific hardware and software used are listed below:

- CPU the Intel Core Ultra 9 185H.
- Intel Arc Graphics as the integrated GPU.
- Intel AI Boost NPU.
- NVIDIA GeForce RTX 4060 Laptop (8GB VRAM).
- RAM 32 GB DDR5.
- OS Kubuntu 25.10 with the linux kernel 6.17.

5.1 Homogeneous Workloads

The first set of tests are on homogeneous workloads, where the scheduler process batch of identical matrix multiplication tasks, we analyzed three different matrix sizes: 1024x1024, 2048x2048, and 4096x4096, all the data used are with 32 bit precision.

In this case, we compare the Static Partitioning logic against the Dynamic logic and for the baseline the CUDA only logic will be used.

5.1.1 Matrix: 1024x1024

In this case, we are dealing with small matrices, where the biggest overhead is certainly the latency of moving data between memories. The results of the logic execution are presented below:

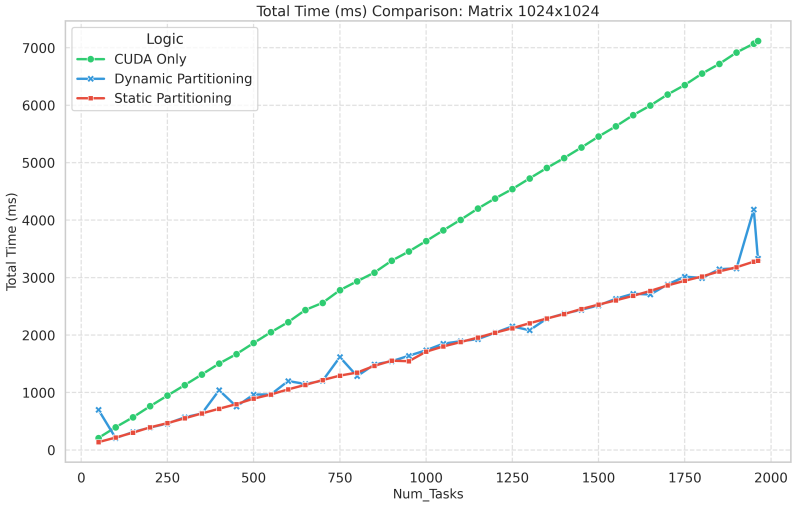

Figure 5.1: Time Comparison 1024

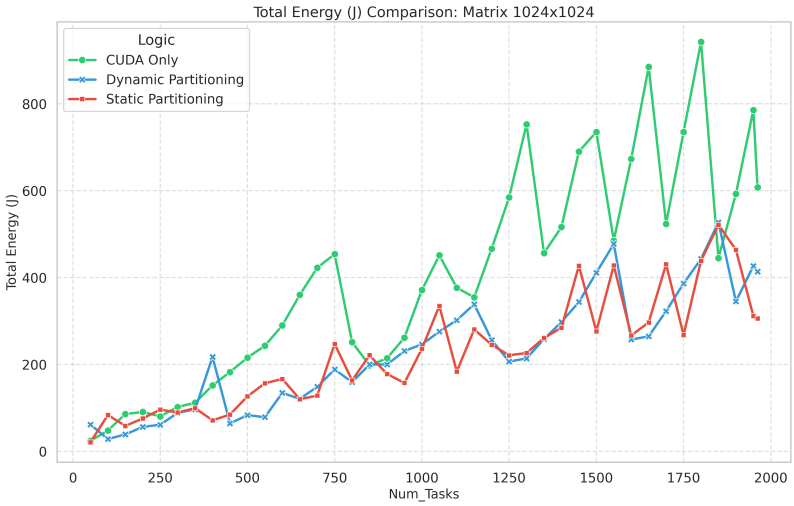

Figure 5.2: Energy Comparison 1024

As we can see from Figure 5.1, with these small tasks, the static logic and the dynamic logic perform almost in the same way, both clearly outperform the single CUDA card by using multiple accelerators in parallel. An average speedup of 2% was observed for the static logic, and 2.10% for the dynamic logic, compared to the CUDA logic. The graph in Figure 5.2 is very interesting, where we can clearly see how the two strategy manage to stay below the total consumption of the CUDA

logic (for energy metrics, see Sec. 4.3.7). This is a very significant result because it shows how these different accelerators, even if they are not as fast as the CUDA card, can still be efficient; and if used in parallel, they lead to an increase in the total system efficiency. In this case, the metric efficiency was calculated as Tasks per Joule, and we obtained:

- Cuda only: 2.62 Tasks/J
- Dynamic: 4.46 Tasks/J
- Static: 4.24 Tasks/J

Furthermore, to investigate the performance of the two logics, graphs were made to see how the tasks are distributed among the various accelerators, and how much time the accelerators actually work during the whole makespan.

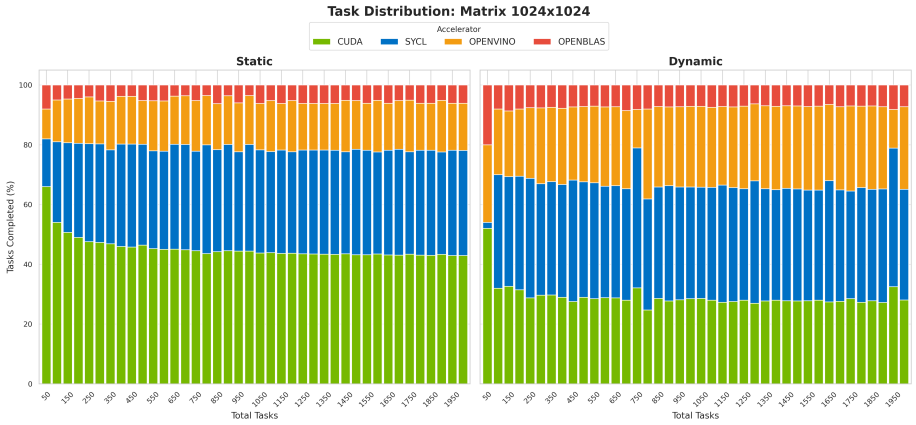

Figure 5.3: Task Distribution 1024

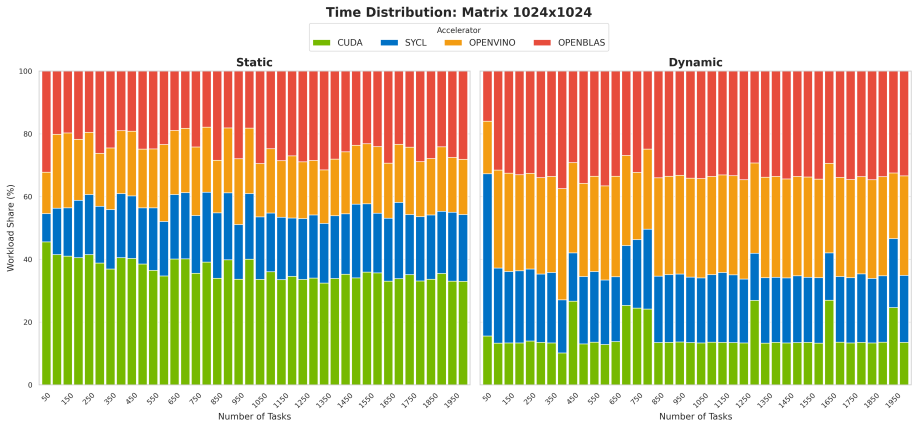

Figure 5.4: Time Distribution 1024

Here we can see how the two logics fundamentally differ in their operation. The greedy nature of the dynamic logic tends to make the accelerators work more con-

tinuously in this case of small tasks, managing to "spread" the work better among the accelerators. As we can note from Figure 5.3, it is clear that the static logic underestimated the performance of the non CUDA accelerators, and this led to a higher number of tasks on NVIDIA card, meanwhile, the dynamic logic managed, thanks to its greedy nature, to make the other accelerators work much more evenly, and this indeed led to a bit more faster times. In fact, as we can see from the graph in Figure 5.4, the dynamic logic definitly make more use of the other accelerators over the CUDA card. This is due to the fact that they are slightly faster in this smaller tasks, having much less memory overhead.

5.1.2 Matrix: 2048x2048

In this scenario, the workload increases significantly. The matrix size is large enough that the computational cost starts to increase over the data transfer overhead. The results of the logic execution are presented below:

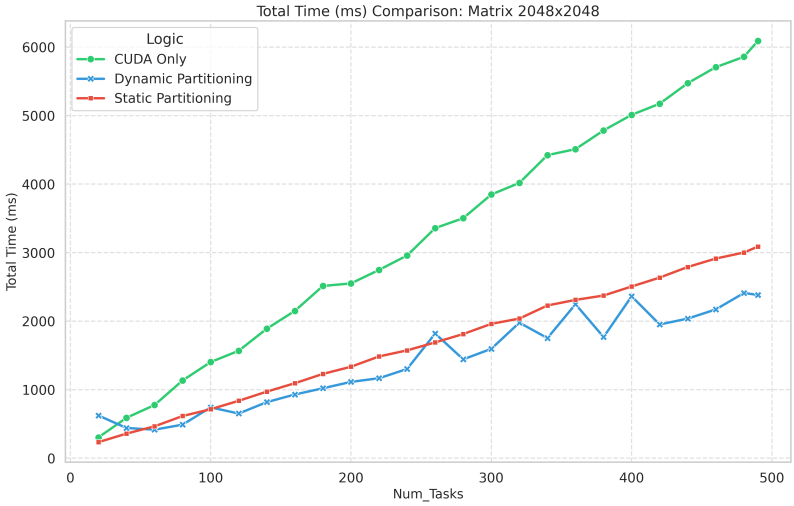

Figure 5.5: Time Comparison 2048

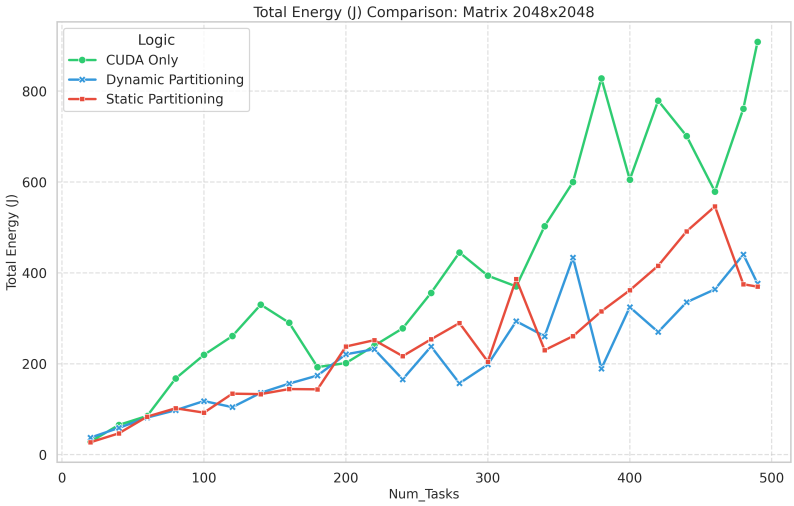

Figure 5.6: Energy Comparison Matrix 2048

As shown in Figure 5.5, both strategies continue to outperform the single CUDA baseline, validating the benefit of parallel execution even in this kind of workload. However, a clear different appears between the two logics. The dynamic strategy achieves an average speedup of 2.21x (peaking at 2.71x), while the static strategy lags slightly behind with an average speedup of 1.90x. This indicates that the dynamic logic is 14% faster on average for this specific workload.

Regarding energy efficiency (Figure 5.6) the heterogeneous execution significantly reduces the total energy consumption compared to the CUDA only approach, thanks to the reduction in total execution time. The dynamic strategy proves to be the most efficient, consuming only 218 J on average per batch, compared to 430 J for CUDA Only.

The efficiency metric show this gain:

- **CUDA Only**: 0.66 Tasks/J
- **Dynamic**: 1.14 Tasks/J
- **Static**: 1.00 Tasks/J

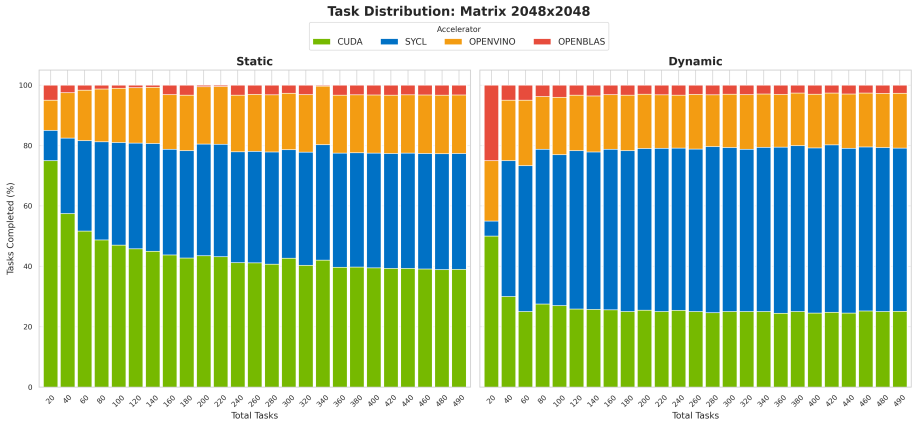

Figure 5.7: Task Distribution: Matrix 2048x2048

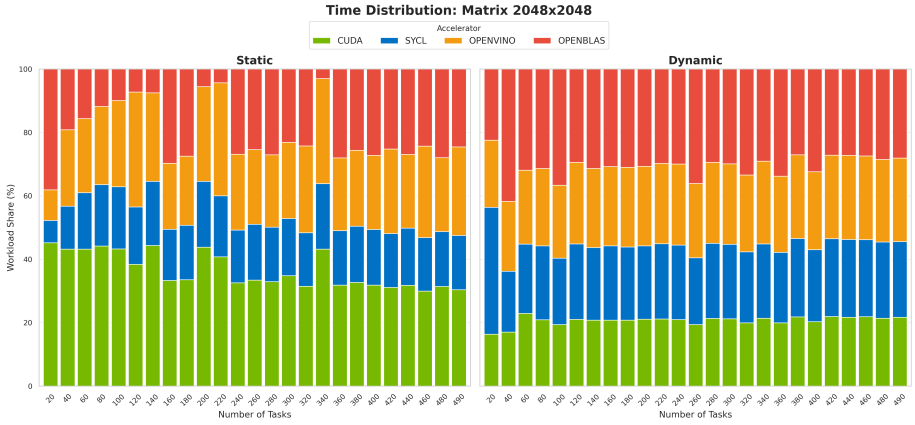

Figure 5.8: Time Distribution: Matrix 2048x2048

Figure 5.7 reveals again the cause of the performance difference. The static strategy assigns a large portion of tasks to the CUDA GPU and a smaller share to the other accelerators. While safe, this distribution appears to underutilize the other accellerators.

However, the dynamic strategy as already said, adapts in real time, so as seen in the Figure 5.8, the dynamic logic keeps the CPU, the intel GPU and NPU work more evenly.

5.1.3 Matrix: 4096x4096

This scenario expose a heavy workload on the scheduler, the computational cost now should dominate the ovehread of data transfer. The results of the logic execution are presented in Fig. 5.9 and Fig. 5.10:

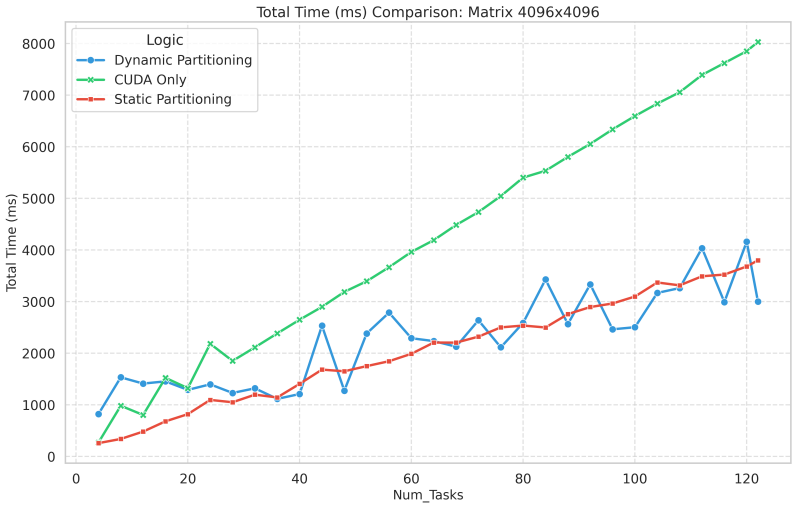

Figure 5.9: Time Comparison 4096

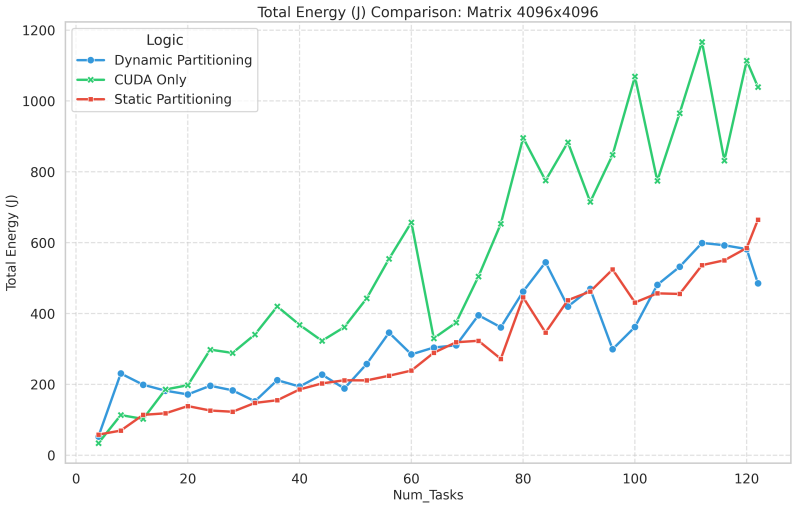

Figure 5.10: Energy Comparison 4096

As shown in Figure 5.9 both strategies still significantly outperform the single CUDA baseline. However, the static strategy show its superiority in this specific scenario,

achieving an average speedup of 2.00x (peaking max at 2.90x) and the dynamic strategy follows with an average speedup of 1.78x (peaking at 2.67x). Ultimately, the static logic proves to be 12.5% faster on average compared to the Dynamic one.

Regarding efficiency (Figure 5.10), the results still shows that despite activating all available accelerators, the parallel execution reduces the total energy consumed, the CUDA-only execution consumes on average 568.70 J, while the heterogeneous strategies drop this to roughly 330-340 J.

- **CUDA Only**: 0.11 Tasks/J
- **Dynamic**: 0.18 Tasks/J
- **Static**: 0.18 Tasks/J

This represents a +61% efficiency gain for the static strategy over the baseline: using the CPU and NPU alongside the GPU for heavy tasks actually saves energy because the entire system finishes the job much faster.

To understand why the Static logic outperforms the Dynamic one here, we analyze the task distribution:

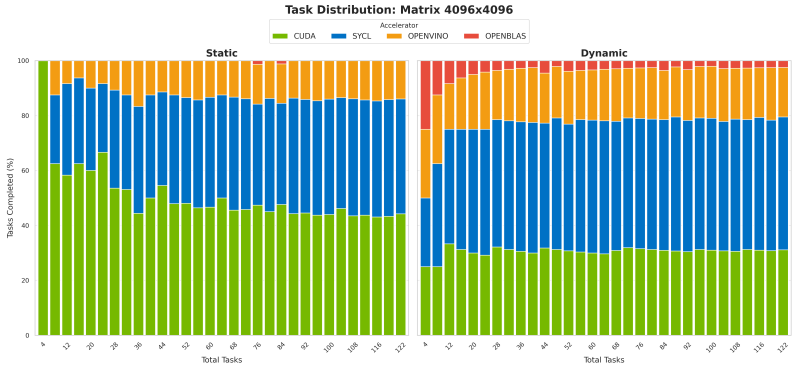

Figure 5.11: Task Distribution 4096

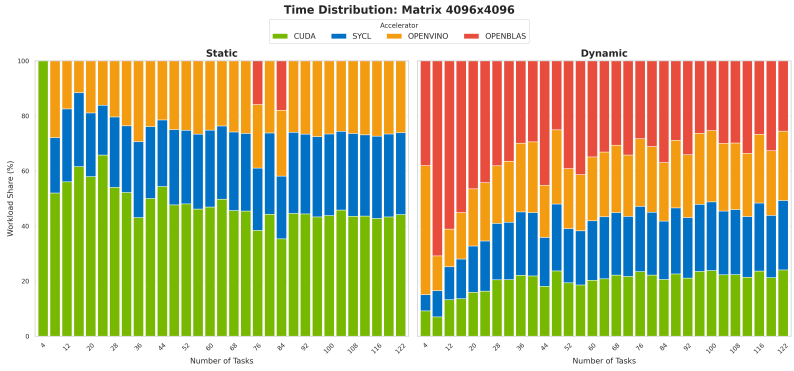

Figure 5.12: Time Distribution 4096

Figure 5.11 shows that the static strategy assigns the vast majority of tasks to the powerful CUDA GPU and SYCL, minimizing the load on slower devices, however, the dynamic logic, due to its greedy nature, assigns a slightly larger portion of tasks to the CPU and OpenVINO (as seen in Figure 5.12). For very large matrices, the time difference between the GPU, NPU and mostly CPU is massive, so if the dynamic logic assigns the last few tasks to a slow device while the GPU finishes early, the entire makespan is delayed waiting for that slow device to complete. The static strategy with all the logics and heuristcs avoids this by predicting the velocity of the accellerators and restricting the slow devices to a more smaller amount of work, achieving the highest speedup. This clearly shows how the static approach have his benefit in a lot of workload in this heterogeneous scenerio.

5.1.4 Batch Comparison

We now present the following graphs to illustrate the impact of batch size on the execution times of the static logic:

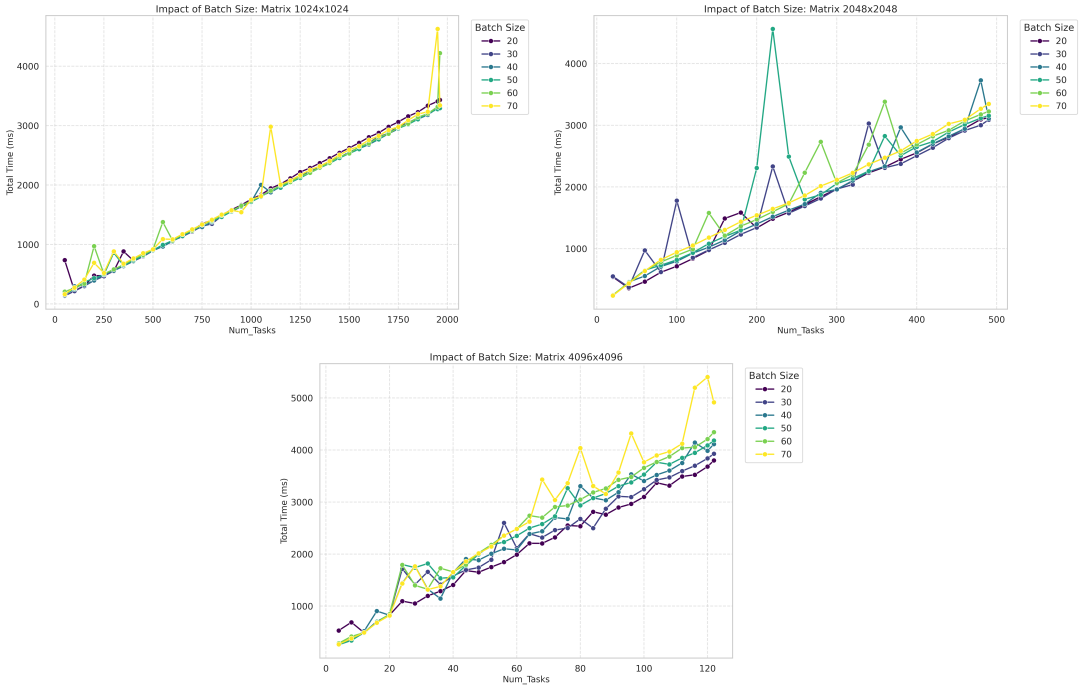

Figure 5.13: Batch Size Analysis

As is evident from the graphs in Fig 5.13, a relatively small batch size specifically ranging from 20 to 30 proved to be the most suitable for the selected algorithm and the dataset utilized across each workload. Conversely, it is evident that larger batch sizes frequently result in inaccurate predictions, while inevitably introducing additional overhead to the coordinator adding more time beetween the dispatch of the tasks to the workers.

5.2 Heterogeneous Workloads

5.2.1 Static Heterogeneous

This test involves streams of matrices with random dimensions (from 512 to 4096). This represents the most realistic scenario for a general-purpose system, where tasks of varying complexity arrive at the scheduler.

The results of the execution are presented in Fig. 5.14 and Fig. 5.15:

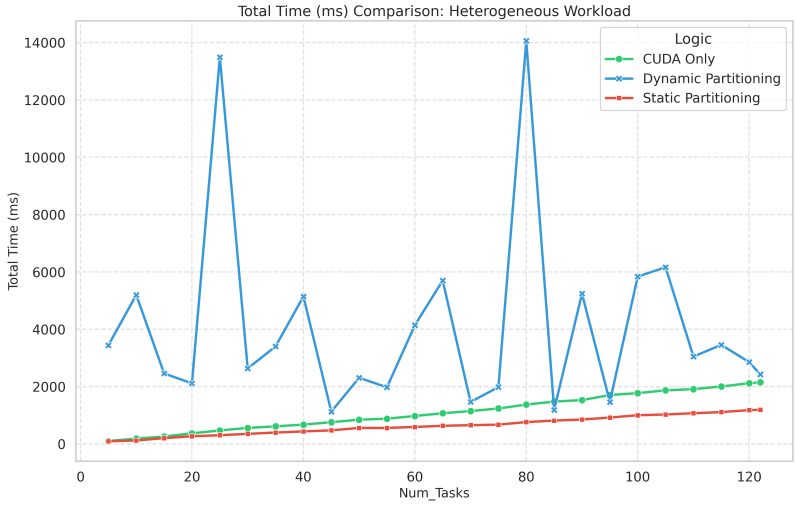

Figure 5.14: Time Comparison: Heterogeneous Stream

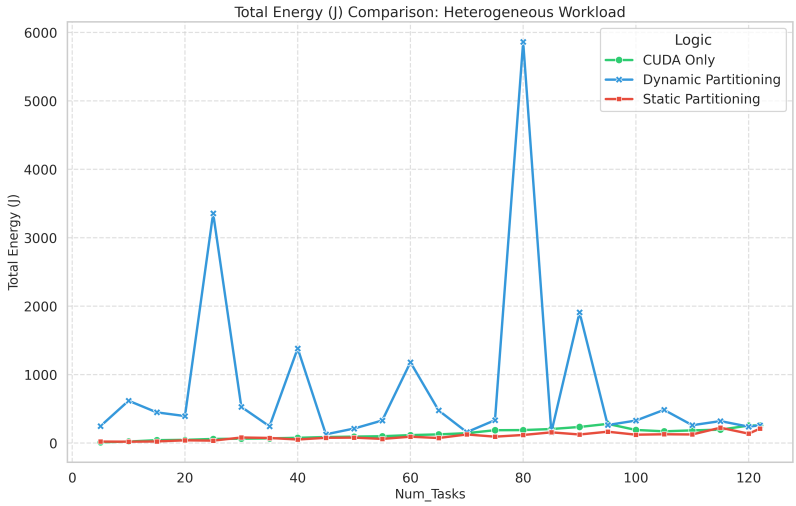

Figure 5.15: Energy Comparison: Heterogeneous Stream

As shown in Fig. 5.14, the difference between the strategies is this case is drastic. The static hetero strategy as the red line proves to be extremely robust, consistently outperforming the single CUDA baseline as the green line. It achieves an average

speedup of 1.65x (peaking at 1.85x), demonstrating how this logic achieves a consistent speedup increase while orchestrating the diverse accelerators to handle the heterogeneous workload.

In contrast, the dynamic strategy as the blue line fails to maintain consistent execution times this type of workload. With an average speedup of only 0.42x, it is significantly slower than using the discrete GPU alone. This failure is due to its greedy nature: without looking ahead, it treats a 512×512 matrix and a 4096×4096 matrix as identical tasks, and consistently fails to distribute them efficiently to achieve maximum performance.

The energy analysis in Fig. 5.15 mirrors the performance gap already observed:

- **Static Strategy**: Consumes on average 112 J, achieving a -17% energy saving compared to the average CUDA baseline (136 J).
- **Dynamic Strategy**: Due to the extremely long execution times, the energy consumption skyrockets to an average of 805 J, with a +490% increase over the baseline.

To better understand these results, we analyze the distribution graphs in Fig. 5.16 and Fig. 5.17:

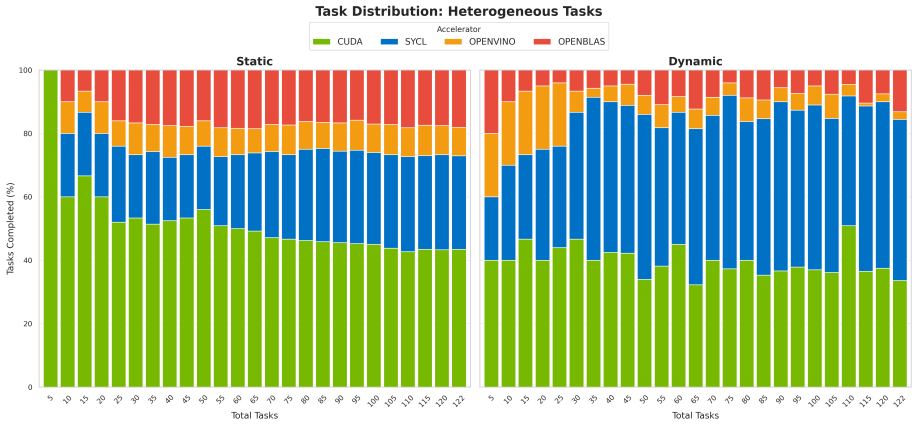

Figure 5.16: Task Distribution: Heterogeneous Stream

Regarding the task distribution (see Fig. 5.16), the graph doesn't describe the full scenario since every task has a different weight. However, for the static strategy, we can observe that the number of tasks assigned to each accelerator remains proportional as the total number of tasks increases. This confirms that the strategy is consistent in its decision making logic.

In contrast, the dynamic strategy shows a completely chaotic behavior. Since tasks are assigned to whichever device is free, the distribution varies unpredictably. Unlike the homogeneous tests where the dynamic distribution was relatively stable with

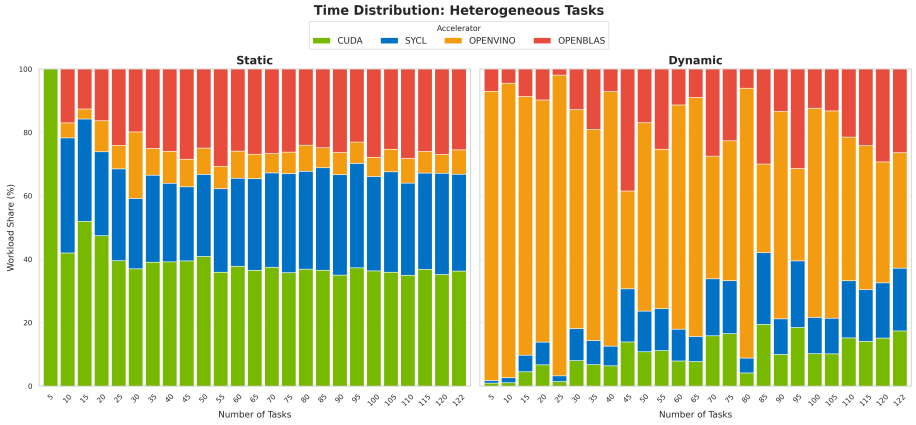

Figure 5.17: Time Distribution: Heterogeneous Stream

smaller tasks, here the randomness of task sizes combined with greedy scheduling leads to a random assignment that changes with every run.

As seen in the Time Distribution (Figure 5.17), the CPU and OpenVINO bars are massive. This indicates that the scheduler blindly assigned computationally heavy tasks (e.g., a 4096×4096 matrix) to slow devices simply because they were "free"" at that moment. This creates a massive bottleneck where the powerful GPU finishes its work quickly and sits idle for seconds, waiting for the CPU to finish a single large matrix.

This result conclusively proves that for heterogeneous streams of tasks, smart scheduling with performance prediction is mandatory in this kind of workloads. A simple greedy approach is insufficient and leads to severe performance degradation.

5.2.2 Batch Comparison

We now present the following graph in Fig. 5.18 to illustrate the impact of batch size on the execution times of the static logic under heterogeneous workload conditions:

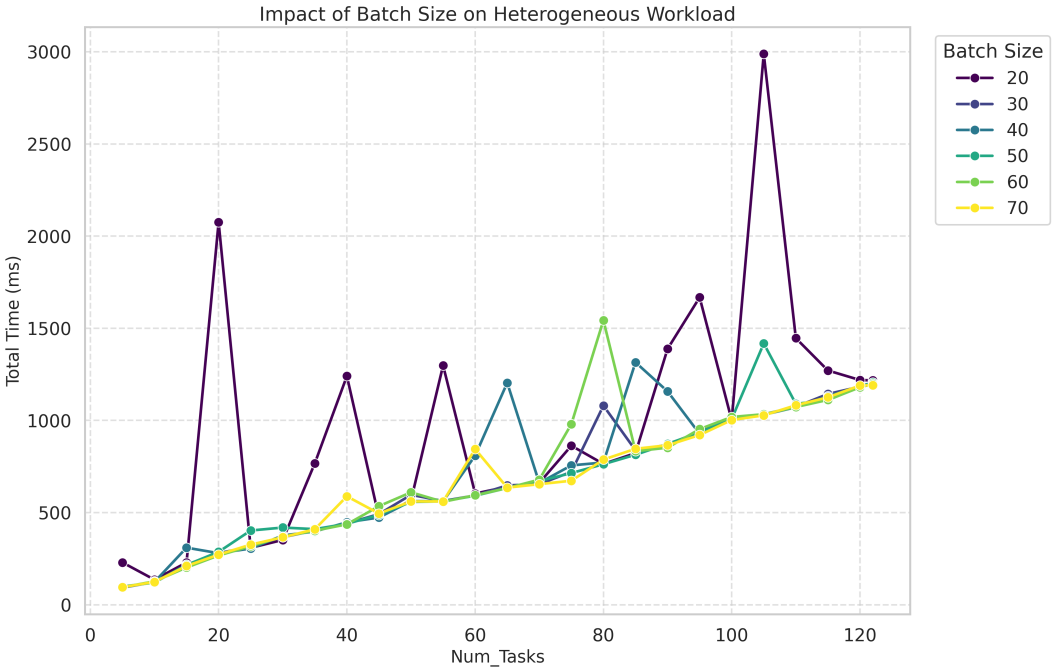

Figure 5.18: Heterogeneous Batch Size

As is evident from the graph in Fig 5.18, a relatively large batch size, specifically ranging from 60 to 70, proved to be the most suitable for the static hetero algorithm. Conversely, it is evident that smaller batch sizes (e.g. 20) frequently result in highly unstable execution times, introducing load imbalance due to the high variance of task duration. Larger batches effectively smooth out these variations, allowing the static ratios calculated within the algorithm to converge towards a more efficient distribution of the task, effectively respecting the planned times.

5.2.3 Large matrix split

This scenario involves decomposing a single very large matrix multiplication (where M grows up to 230,000 rows) into smaller sub-tasks (chunks) to fit within device memory limits and distribute the computation.

To provide a fair and meaningful baseline, a simplified version of the splitting logic was implemented. This strategy performs the same chunking operation required to handle the large matrix dimensions but dispatches all chunks exclusively to the NVIDIA GPU (the code can be seen in Appendix **??**).

Furthermore, for this specific test case, the OpenVINO backend was excluded. Pre-

liminary tests showed that OpenVINO's performance degrades when handling extremely "rectangular" matrices (very high M, fixed N,K), making it an unsuitable for this specific splitting workload. Our strategy therefore in this test leverages CUDA, SYCL, and OpenBLAS.

The following graphs (Fig. 5.19) illustrate the comparison between the Hybrid Split strategy and the CUDA Only baseline:

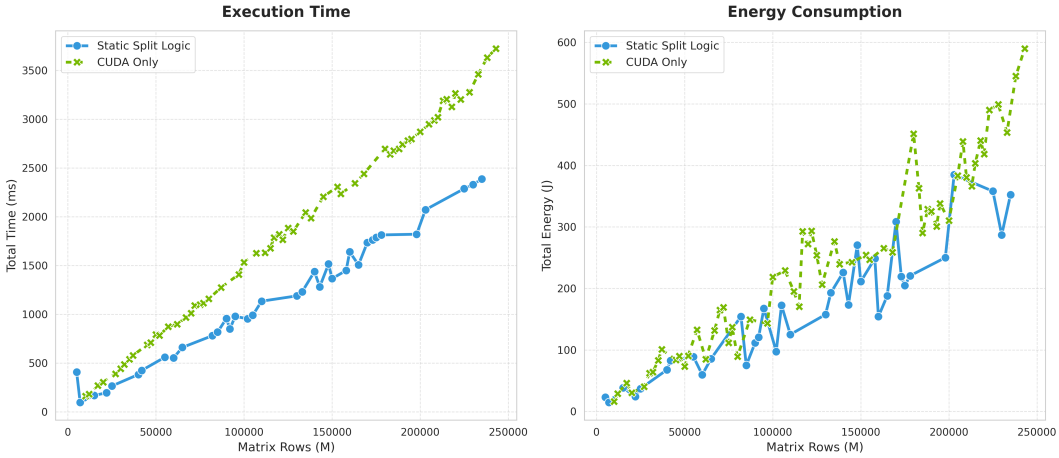

Figure 5.19: Time and Task Analysis

The execution time comparison on Fig. 5.19 on the left clearly demonstrates the advantage of the Hybrid split approach. The split logic consistently remains below the green line (CUDA Only), indicating a lower total time to completion across the entire range of matrix sizes. The hybrid strategy achieves an 1.49x average speedup compared to using the GPU alone. In specific configurations the speedup reaches a peak of 2.46x, demonstrating that the combined throughput of the CPU and iGPU can more than double the system's performance for certain problem sizes.

Consumptions wise, the Hybrid strategy also demonstrated superior energy efficiency in some cases (see Fig. 5.19 on the right): on average, the large static split approach reduced energy consumption by 21% compared to the CUDA baseline. Furthermore over the entire benchmark, the hybrid strategy consumed 38 J less on average than the 52 J required by the CUDA only execution. This result highlights that finishing the task faster, even by powering up the CPU and iGPU, is often more energy efficient than keeping the discrete GPU active for a longer duration.

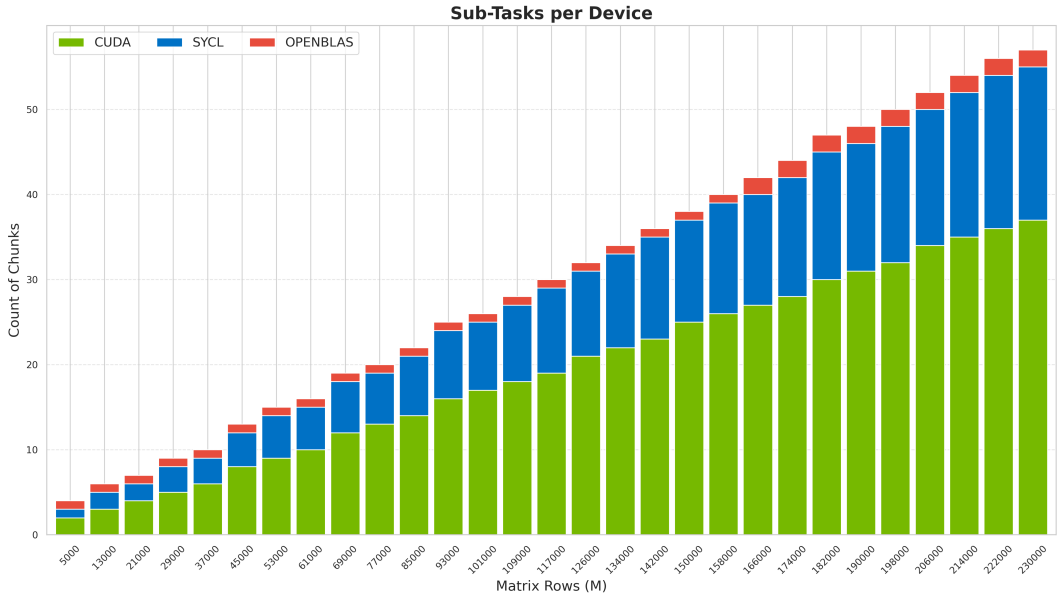

Figure 5.20: Chunk Distribution Analysis

The load balancing graph on Fig. 5.20 provides insight into how the work was distributed. As expected, the powerful CUDA GPU handles the majority of the chunks (approximately 50% or 60% of the workload). The remaining of the work is successfully offloaded to the iGPU and CPU backends. This is the direct cause of the observed speedup: by keeping all accelerators active, the system completes the massive calculation faster than the single GPU could.

Appendix A

Questa è un'appendice

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Bibliography

[1] Intel Corporation. *Intel Arc Graphics Xe-LPG Architecture*, 2023. https://www.intel.com/content/www/us/en/docs/oneapi/optimization-guide-gpu/2023-2/intel-xe-gpu-architecture.html.

[2] Intel Corporation. *Intel Core Ultra Processors (Series 1)*, 2024. https://www.intel.com/content/www/us/en/products/sku/236849/intel-core-ultra-9-processor-185h-24m-cache-up-to-5-10-ghz/specifications.html.

[3] Intel Corporation. Intel oneapi: A unified programming model, 2024. https://www.intel.com/content/www/us/en/developer/tools/oneapi/overview.html.

[4] Intel Corporation. *Intel's Neural Processing Unit*, 2024.

[5] Intel Corporation. *Openvino Documentation*, 2025. https://docs.openvino.ai/2025/index.htm.

[6] Brian Kernighan and Dennis Ritchie. *The C Programming Language*. Prentice Hall, 1988.

[7] Kitware, Inc. *CMake Documentation*, 2025. https://cmake.org/cmake/help/latest/.

[8] Klaus Hinum. *NVIDIA GeForce RTX 4060 Laptop GPU Spec*, 2022. https://www.notebookcheck.net/NVIDIA-GeForce-RTX-4060-Laptop-GPU-Benchmarks-and-Specs.675692.0.html.

[9] NVIDIA Corporation. *Cublas*, 2025. https://docs.nvidia.com/cuda/cublas/index.html.

[10] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2025. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[11] NVIDIA Corporation. *NVIDIA Management Library (NVML)*, 2026. https://developer.nvidia.com/management-library-nvml.

[12] OpenBLAS Team. Openblas github, 2024. https://github.com/OpenMathLib/OpenBLAS.

[13] Steve Rennich. *CUDA C++ Streams and Concurrency*, 2024. https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf.

[14] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 2013.

[15] The Khronos Group. Sycl overview and specification, 2024. https://www.khronos.org/sycl/.

[16] Zhang Xianyi. Openblas, 2025. http://www.openmathlib.org/OpenBLAS/.